

# On Deniable Authentication against Malicious Verifiers

Rune Fiedler<sup>1</sup> and Roman Langrehr<sup>2</sup> 

<sup>1</sup> Technische Universität Darmstadt

[rune.fiedler@cryptoplexity.de](mailto:rune.fiedler@cryptoplexity.de)

<sup>2</sup> ETH Zurich

[roman.langrehr@inf.ethz.ch](mailto:roman.langrehr@inf.ethz.ch)

**Abstract.** Deniable authentication allows Alice to authenticate a message to Bob, while retaining deniability towards third parties. In particular, not even Bob can convince a third party that Alice authenticated that message. Clearly, in this setting Bob should not be considered trustworthy. Furthermore, deniable authentication is necessary for deniable key exchange, as explicitly desired by Signal and off-the-record (OTR) messaging.

In this work we focus on (publicly verifiable) designated verifier signatures (DVS), which are a widely used primitive to achieve deniable authentication. We propose a definition of deniability against malicious verifiers for DVS. We give a construction that achieves this notion in the random oracle (RO) model. Moreover, we show that our notion is not achievable in the standard model with a concrete attack; thereby giving a non-contrived example of the RO heuristic failing.

All previous protocols that claim to achieve deniable authentication against malicious verifiers (like Signal’s initial handshake protocols X3DH and PQXDH) rely on the Extended Knowledge of Diffie–Hellman (EKDH) assumption. We show that this assumption is broken and that these protocols do not achieve deniability against malicious verifiers.

**Keywords.** Deniability, Designated verifier signature, Random oracle model, Rogue key attacks

## 1 Introduction

In an offline world, speaking convinces the immediate listener of the authenticity of the spoken word but not necessarily a third party, to whom the listener relays the conversation later. In fact, this can be the reason why somebody *dares* to speak in the first place, e.g., in case of health issues, whistleblowing, or moral but illegal protests. We call this property *deniable authentication*, where one party can authenticate a message to the intended recipient and deny its contents to all other parties.

Formalizing deniable authentication in the digital realm has been a subject of research for more than three decades, e.g., see works by Dolev, Dwork, and Naor [25], Dwork, Naor, and Sahai [29], and Di Raimondo and Gennaro [22].

Several primitives provide deniable authentication, including: 1) designated verifier signatures, introduced by Jakobsson, Sako, and Impagliazzo [42], where a signer *designates* a signature to a specific verifier, who cannot transfer this authentication to a third party; 2) ring signatures, introduced by Rivest, Shamir, and Tauman [57], which guarantee that 1 *out of*  $n$  ring members authenticates a message; 3) split KEMs, introduced by Brendel, Fischlin, Günther, Janson, and Stebila [14], dubbed authenticated KEMs (AKEMs) by Alwen, Blanchet, Hauck, Kiltz, Lipp, and Riepel [1], which allow decapsulation of an encapsulated shared secret in a manner that deniably authenticates the encapsulator.

Deniable authentication has since been transferred to the protocol layer. In particular, Di Raimondo, Gennaro, and Krawczyk [23] have introduced deniability for key exchange. Chat applications and messaging protocols are particularly interested in deniability. Off-the-record (OTR) messaging prominently discussed deniability [11, 69, 70]. Signal, inspired by OTR, also aims for deniability of its initial handshake protocol. Previously, they deployed X3DH [51], which Vatandas, Gennaro, Ithurburn, and Krawczyk [71] showed to be deniable under their novel Extended Knowledge of Diffie-Hellman Assumption (EKDHA). Looking ahead, in Section 5.2 we show that the EKDHA is broken. For the post-quantum replacement protocol PQXDH [45], Fiedler and Janson [32] have proven slightly worse deniability properties that additionally require plaintext awareness of the used KEM. Several post-quantum replacement protocols for Signal’s initial handshake were proposed [35, 13, 24, 18] and all proved some form of deniability. We refer the reader to [32, Appendix A] for a more detailed discussion of different notions of deniability for key exchange.

### 1.1 Malicious verifier keys

In the setting of deniable authentication, Alice wants to convince Bob, without Bob being able to convey his conviction to Charlie. If Alice was able to trust Bob not to transfer his belief to Charlie (and if Alice was assured that Bob cannot be coerced into cooperating with Charlie), then we would not need special precautions to ensure deniability in the first place. Hence, while Alice wants to authenticate a message to Bob, she does not trust him. As a logical consequence, Alice should not trust Bob to generate his keys honestly. We apply this idea to designated verifier signatures (DVS), which we consider to be the most basic primitive out of those mentioned above<sup>3</sup>.

A DVS that retains unforgeability and deniability<sup>4</sup> against malicious verifier keys has several uses. A whistleblower can deniably authenticate her message, even if the receiving journalist is malicious, compromised, or negligent. In more extreme cases, law enforcement might pose as a journalist to gather incriminating material on a whistleblower for a court trial. Given that regimes around the

<sup>3</sup> Split KEMs and AKEMs additionally ensure confidentiality of the shared secret, and ring signatures are more generic since they can deal with more than two users.

<sup>4</sup> In the rest of the paper we refer to the deniability property as “source hiding”, cp. Definition 17.

world are prosecuting whistleblowers, this does not appear far-fetched. Hence, we cannot trust a verifier to generate keys honestly.

Furthermore, in a chat application users interact with many other users to which they have different levels of trust. And even if one of the chat partners turns out to be malicious, users expect deniability in all chats, including the chat with the malicious peer, and confidentiality in all remaining chats. We study security notions for DVS that provide these assurances.

## 1.2 Related concepts and works

The concept of verifier keys being maliciously generated has already appeared several times in the literature. Closest to our field of study, Shim [62] and Zhang, Au, Yang, and Susilo [73] have studied multi-DVS against malicious verifier keys, dubbed *rogue key attacks*. An unforgeable signature scheme can exhibit unexpected behavior against maliciously generated keys, such as one signature verifying messages under two distinct public keys, one signature verifying two distinct messages, or a signature not guaranteeing that the signer knows the message. These Beyond UnForgeability Features (BUFF) were defined and compiled by Cremers, Düzl , Fiedler, Fischlin, and Janson [20], and further refined in subsequent work [5, 27, 26, 28]. Looking at KEMs, Cremers, Dax, and Medinger [19] studied whether shared secrets, public keys, and ciphertexts may be bound to each other (e.g. a shared secret may only stem from decapsulating one particular ciphertext). One of their settings allows maliciously generated public keys as well.

A different form of deniability can be achieved with time-deniable or epochal signatures, where, informally speaking, signatures provide authenticity only for a certain time-span and can be forged afterward. Different variants of this idea have been studied in [64, 39, 2, 7]. Since the verifier is completely passive in this setting and does not even issue a public key, there is no need to model a malicious verifier.

## 1.3 Our contributions

We propose new security definitions for DVS that provide unforgeability and deniability against malicious verifiers. Similar notions for unforgeability were already considered by previous works [62, 73], but the notion for deniability is novel. We also propose a construction satisfying our new definitions in the random oracle model. Furthermore, we show that our new deniability notion cannot be achieved in the standard model by giving a concrete attack. In particular, this shows that instantiating our random oracle construction with a concrete hash function results in an insecure scheme. To the best of our knowledge, the only way to achieve a similar security notion so far relied on the EKDH assumption<sup>5</sup>, which we show to be broken.

<sup>5</sup> The deniability of Signal’s initial handshake protocols X3DH and PQXDH rely on the EKDH assumption.

**Model.** Recall that the standard notion of deniability for DVS guarantees that a valid signature generated with the signer’s secret key (and the verifier’s public key) is indistinguishable from a simulated signature, that was generated with the signer’s public key and the verifier’s secret key.

To capture that verifiers might generate their keys maliciously, we aim for a security notion where the adversary chooses the verifier’s public key. In this setting, there might not be a matching secret key for the verifier’s public key. Thus, we need a different approach to simulate signatures. Clearly, the simulation should require some secret that only the (potentially malicious) verifier possesses, since otherwise unforgeability could no longer hold. Moreover, everybody should be convinced that the verifier actually possesses this secret: Otherwise, if the verifier could convince a judge of not possessing the secret, the judge would be convinced that a signature must have been generated by the purported signer. In this work, we propose to use the *random coins* used to generate the verifier’s public key (without making any restrictions on *how* the verifier generated this public key) as such a secret. Clearly, a classical verifier (without quantum capabilities) must have known these random coins when generating the public key and it is impossible to convince a third party that he deleted these random coins, since the verifier can always keep a copy of the random coins.

More concretely, we propose an extractor-based definition where the simulated signature is generated by an extractor that can depend on the adversary and is given the adversary’s random coins. Following [55], the extractor should only have capabilities that a verifier actually has in the real world. In particular, this excludes knowing the trapdoor to a common reference string (crs) or the ability to program a random oracle (RO). The extractor might still simulate the adversary in its head and change the crs or program the RO in these simulations, but it has to output a signature that is indistinguishable in the *real world*, where the extractor does not possess these capabilities.

The security notion guarantees that no adversary can tell apart real signatures from those generated by the extractor for an honestly generated signer public key and a verifier public key and message chosen by the adversary.

*Identities.* Unfortunately, the model described so far is very unrealistic, because in the real world it is fairly easy to generate a public key without knowing the random coins used to generate it. One way to achieve this is to copy the public key of another user.

We avoid this attack by introducing identities, which are meaningful unique identifiers of each user, e.g., an e-mail address, a phone number, or a username in a chat application. Verifier keys are generated with respect to an identity and the signer signs messages with respect to a verifier public key and a verifier identity. Signing only has to output valid signatures (for which correctness and unforgeability hold) if the verifier identity matches the one used to generate the verifier public key.

Looking ahead, our ROM construction prevents these key copying attacks by outputting a dummy value when the identity the signer intended to sign for does not match the identity used for generating the verifier public key.

*Auxiliary input.* Of course, there might be different ways to obtain random data without knowing the random coins in the real world. For example, in the real world a verifier has access to public keys or signatures of other, unrelated deployed cryptosystems. A malicious verifier could use such data to generate his own public key.

If the adversary sampled this data on its own, it would know the random coins and therefore also the extractor would get access to these random coins, which is not realistic. Therefore, we have to provide the adversary (and thus also the extractor) with auxiliary input (*common auxiliary input model*<sup>6</sup>).

Of course, we now have to avoid that the auxiliary input just contains a verifier public key for our DVS. Otherwise, a malicious verifier could just use the public key from the auxiliary input. An extractor would then have to forge a signature for a signer and verifier public key for which it knows no secret. The existence of such an extractor would contradict the unforgeability of the signature scheme.

We avoid this by using a (trusted) setup for our DVS that generates public parameters that are given as input to all other algorithms of the DVS. The auxiliary input is not allowed to depend on the public parameters. This also shows that our security notion cannot be achieved in the plain model (without trusted setup) and requires at least the uniform random string (urs) model, where the public parameters are uniformly random bits.

Finally, an adversary can realistically also get some inputs depending on the public parameters, such as signer public keys, verifier public keys for different identities (and possibly also the corresponding secret keys, modeling users who are negligent in keeping their secret keys safe), and signatures. As a first step in this line of research we keep this to a minimum and propose a notion we call *simplified malicious source hiding* where the adversary gets access to an oracle for generating verifier public keys.

Prior work [23] (and subsequently [71, 32]) use auxiliary inputs to model data that a receiver is able to obtain that is *helpful* to simulate a protocol transcript. This is very different to our use of auxiliary inputs, where the auxiliary input is only to the *disadvantage* of the simulator. Formally, the difference is that our security game asks for security to hold for *all* auxiliary inputs  $\mathbf{aux} \in \{0,1\}^*$ . In particular, this includes the case where  $\mathbf{aux}$  is the empty string and thus our security notion is stronger than the version without any auxiliary input. In contrast, e.g. [23], only demand security for *some* auxiliary inputs (all auxiliary inputs that form a valid protocol transcript) and this can be weaker than the variant without any auxiliary input.

---

<sup>6</sup> Here, “common” refers to the adversary and the extractor getting the same auxiliary input. If we allow the extractor to get a different (existentially quantified) auxiliary input, it would be much easier to achieve our security notion, but that model is not realistic: For example, if the adversary’s auxiliary input contains a public key of a different cryptosystem, the simulator’s auxiliary input can contain the secret key. This is not realistic, because it is not feasible for the real receiver to retrieve the secret key and the simulator should model the real receiver’s capabilities.

We discuss possible restrictions on the auxiliary input in [Section 1.4](#).

*Limitations.* The approach of using the random coins as the verifier’s secret still has a few limitations.

1. The verifier might not know the random coins if he generated the key with the help of someone else (for example with multi-party computation). In this case the knowledge of the random coins might be spread among several people (or devices). On the one hand, this seems unavoidable (unless we assume a trusted party or trusted device generating the verifier’s key). On the other hand, we think this is not a severe limitation: If several users collaborate to generate the verifier public key, it might be hard to convince a judge that they did not also collaborate to produce the randomness.
2. Quantum capabilities (or other physical processes) might enable the verifier to produce a verifier key and prove that he does not know the corresponding random coins.

*New security definitions.* We present our new security notions in [Section 3](#) and show separations from existing security notions (that do not capture maliciously generated keys) in [Appendix A](#).

**Non-solutions.** Before describing our solutions, we describe a few natural approaches that fail to achieve deniability against malicious verifiers.

*Knowledge-type assumptions.* Since our security notion involves an extractor, one might be tempted to use knowledge-type assumptions like the knowledge of exponent assumption [\[21\]](#). However, most of these assumptions imply extractable one-way functions (EOWF), which cannot be secure in the common auxiliary input model [\[9\]](#).

Vatandas et al. [\[71\]](#) propose the Knowledge of Diffie-Hellman (KDH) and extended KDH (EKDH) assumption, which are knowledge-type assumptions that do not imply EOWF. They conjecture that these assumptions are hard in the common auxiliary input model. Vatandas et al. and Fiedler and Janson [\[71, 32\]](#) use this assumption to achieve deniability notions for the Signal’s initial handshake protocols X3DH and PQXDH. We show that this assumption is broken by giving a relatively simple attack in [Section 5.2](#). Moreover, the attack can also be extended to these protocols, showing that they do not achieve deniability in a model that allows maliciously generated public keys. We give more details on this attack in [Appendix B](#).

*NIZKPoKs.* A standard technique to deal with maliciously generated public keys is to equip them with a proof of knowledge of the secret key. Informally speaking, this restricts an adversary to public keys from the image of the key generation

algorithm<sup>7</sup> and allows the reduction to extract a secret key for adversarially generated public keys.

To see why this approach does not work in our setting, we have to dig into the definitions of knowledge soundness for non-interactive zero-knowledge proofs of knowledge (NIZKPoKs). There are two incomparable notions in the literature.

The first variant of knowledge soundness allows for extraction of a witness with a trapdoor to the common reference string (crs) of the NIZKPoK. Such an extraction is not helpful for our security notion, because giving the extractor such a trapdoor is unrealistic (as explained in the section on the model).

Another variant of knowledge soundness allows for extraction of a witness from the randomness used to generate the NIZKPoK. This notion is typically considered for succinct non-interactive arguments of knowledge (SNARKs), where the previous notion is unachievable. This form of extraction would work well with our simplified malicious source hiding notion, but since SNARKs (or any NIZKPoK satisfying this type of extractability) cannot be secure in the common auxiliary input model [9], we cannot use them.

**A solution in the ROM.** The starting point of our solution is to equip the public key with a NIZKPoK of the secret key, where the NIZKPoK satisfies the crs trapdoor based extraction. Assume a regular DVS in which we equip all public keys with a NIZKPoK of the secret key. As explained above, this solution alone will not work, because the extractor has no access to a trapdoor for the crs.

However, the extractor can generate its own crs along with a trapdoor and simulate internally the adversary on this new crs. Assume for the moment the adversary generates the DVS verifier public key independently of the crs of the NIZKPoK. The extractor could then use the trapdoor to extract the verifier's secret key and use it to simulate the signature.

The missing ingredient for this approach is a tool to *enforce* that the adversary generates the DVS verifier public key independently of the crs. Our solution for this problem is to pick the crs as the hash of the public key, the verifier's identity and a random string  $s$  that is part of the public parameters of the scheme. Clearly, the adversary then has to query the ROM on these values to generate a valid NIZKPoK for the verifier public key. We include  $s$  in the hash, because then this ROM query cannot be made during the generation of the auxiliary input – which happens independently of  $s$  – and has to be done by the adversary. Since we model identities, this query cannot be made during the generation of any of the verifier keys we provide to the adversary (since they use a different identity).

This idea is reminiscent of the Fiat–Shamir transform [31], that compiles a  $\Sigma$ -protocol to a NIZK by choosing the challenge as a hash of the commit message. This ensures a random challenge generated independently of the commit message.

Finally, the underlying DVS has to satisfy source hiding for all valid verifier public/secret key pairs (instead of an honestly generated one), since the NIZKPoK

<sup>7</sup> Note that such a NIZKPoK does not guarantee anything about the *distribution* of the public key, thus it might still follow a very different distribution than honestly generated public keys.



does not give us any guarantees about the *distribution* from which the key pair was sampled. We show that a DVS satisfying this notion can be built from OR-proofs.

**Impossibility result.** We also show in this work that our simplified malicious source hiding definition cannot be achieved by DVS in the standard model, assuming that indistinguishability obfuscation (iO) [6] exists. With the recent progress on constructing iO [41, 40, 56], it seems very plausible that iO exists. The result is reminiscent of [9]. However, we cannot use their result directly, because our simplified malicious source hiding definition does not imply extractable one-way functions (as far as we know).

Recall that if the auxiliary input would contain a valid verifier public key (without the secret key or any hints on how it was generated), the adversary could simply use this public key in the security game. The extractor, who gets the adversary’s random coins, then gets no useful information how that public key was generated. Hence, simulating a signature is as hard for the extractor as breaking unforgeability of the DVS.

However, verifier public keys can depend on the public parameters, which are not available during generation of the auxiliary input. Hence, we cannot directly include a valid verifier public key in the auxiliary input. Though, we can include an obfuscated program that takes the public parameters as input and outputs a valid verifier public key for the public parameters, which yields essentially the same problem as before. We present this argument in detail in [Section 5.1](#).

This attack also applies to our ROM based construction, when we replace the random oracle with a concrete hash function. Thus, the wide-spread heuristic of replacing a random oracle with a suitable hash function [8] fails here. To see where exactly the heuristic fails, note that in our ROM based construction it is necessary to make a random oracle query to generate a verifier public key (to obtain the crs for the NIZKPoK). However, an obfuscated circuit cannot access the random oracle, therefore this attack does not work for our ROM based scheme. However, if we replace the ROM with an efficiently computable function, the obfuscated circuit can evaluate this function, therefore the attack works.

There have been examples of constructions in the random oracle that become insecure when we replace the random oracle by any concrete hash function prior to this work [16, 34]. However, these constructions have been criticized for being “contrived counter-examples” and they did not stop cryptographers from instantiating ROM schemes with concrete hash functions in practice. Our example for a failure of the ROM heuristic is not contrived. In fact, the authors of this work had the idea for the ROM based construction before becoming aware of this impossibility result and realized the failing of the ROM heuristic only when they tried to resolve the alleged contradiction between the construction and the impossibility result. The authors did not intend to construct a scheme where the ROM heuristic fails.

Concurrently to this work, Khovratovich, Rothblum, and Soukhanov also present a non-contrived counter-example of the random oracle heuristic [44]. Their



counter-example works very different to ours on a technical level. Previous works obtained non-contrived counter examples only for the ideal cipher model [12].

Moreover, several previous works have also introduced security notions that are impossible to achieve in the standard model but achievable in idealized models such as the random oracle model. Examples of this are non-interactive non-committing encryption [52] and simulation-secure functional encryption [54, 10]. However, these works use a simulation-based security notion and the impossibility is due to the so-called “commitment problem” (the simulator has to commit to answers of future queries before it can possibly know how to answer them). This is different to our notion (and the common auxiliary-input model notions for SNARKS [9]), where a concrete attack against the scheme exists in the standard model.

Our impossibility result also rules out the existence of ring signatures that retain anonymity (among the *entire* ring) if at least one public key in the ring was generated maliciously, since such a ring signature would imply a simplified malicious source hiding DVS. Moreover, our impossibility result makes no use of the oracle returning verifier public keys, and therefore applies also to a weaker notion of security which can be achieved without identities.

We want to emphasize here that this impossibility result crucially relies on the public verifiability of DVS. In a privately verifiable DVS, the extractor could generate a signature that does not verify (and therefore does not contradict unforgeability), but is still indistinguishable from a real signature to the adversary.

#### 1.4 Open problems

A compelling question is whether deniability against malicious verifiers can be achieved in a weaker model, that circumvents our impossibility result while still being realistic. A natural restriction is to allow only *efficiently sampleable* auxiliary inputs. However, this does not help to circumvent our impossibility result, because the auxiliary input for our result is efficiently sampleable. Another realistic variant would be to restrict the auxiliary input to contain at least certain information that a receiver can realistically obtain (for example data that is published in a public bulletin board) and that can help the simulator. However, that also does not help to circumvent our impossibility result, since this result holds as long as the auxiliary input can also contain a suitable obfuscated circuit.

The authors of this work are convinced that restricting the auxiliary input to circumvent our impossibility result does not result in a security notion that meaningfully captures deniability with malicious verifiers: The reason is that such a result would have to restrict the auxiliary input to *not* contain a suitable obfuscated circuit. However, a saboteur (e.g. a government that wants to make end-to-end encrypted messaging less attractive) might sample a bit string *aux* such that the DVS is not malicious source hiding with respect to this *aux*. Assume the saboteur publishes *aux* on her website and deletes the random coins used. Now deniability for every other user (except the saboteur herself) is lost, because the verifier could retrieve *aux* from the saboteur’s website and use it to generate his key. Even worse, such an auxiliary input might be generated and published

without any malicious intent. In fact, there are proposed schemes that require as setup to generate an obfuscated circuit that is suitable for our impossibility result and to delete the random coins used to generate it (for example, the constructions from selectively secure universal samplers in [36]).

A more promising approach to circumvent our impossibility result is to consider privately verifiable DVS, since our impossibility result crucially relies on the public verifiability of the DVS. Privately verifiable DVS are sufficient for many applications that require deniable authentication, for example messaging. If this can be answered positively, it would also be desirable to achieve a more realistic notion that gives the adversary all values it can realistically obtain without generating them itself, such as signer public keys and signatures.

Another approach to circumvent the impossibility result is to use an interactive protocol for deniable authentication instead of a DVS. In the interactive setting, the verifier can, for example, use an interactive proof to prove knowledge of his secret key. In contrast to non-interactive proofs, the extraction of a witness can then be done without knowing any secrets (like a trapdoor to a crs) and can be achieved in the presence of auxiliary inputs.<sup>8</sup> In practice, these interactive proofs can also be performed by each verifier once with the PKI instead of running the interactive proof once for every signer. We leave a formal treatment of this approach to future work.

An interesting new research direction is to find new ways of deploying random oracle based schemes, that also work for our DVS in the ROM. For example, an approach could be to use trusted hardware that implements a pseudo-random function with a hard-coded key. Every user of the system then needs such a device with the same key, while the key is not publicly known.

Furthermore, our simplified malicious source hiding definition is not meaningful against quantum adversaries. We leave it as an open question to model source hiding against malicious verifiers in a (post-)quantum setting.

Finally, we leave open to transfer our notion (or a stronger, more realistic version) of deniability against malicious verifiers to more complex primitives and the protocol level, e.g., for authenticated KEMs and key exchange.

## 2 Preliminaries

### 2.1 Notation

We denote the output  $y$  of a deterministic algorithm  $A$  on input  $x$  as  $y \leftarrow A(x)$ , and  $y \xleftarrow{\$} A(x)$  for a probabilistic algorithm or  $y \leftarrow A(x; r)$  with explicit randomness  $r$ . We write  $x \xleftarrow{\$} X$  for (uniformly) sampling from a set or distribution  $X$ , and  $\text{Im}(A)$  for the image of an algorithm  $A$ . We write  $\mathbb{E}_{x \xleftarrow{\$} X} f(x)$  for the expected value of the random variable  $f(x)$  with  $x \xleftarrow{\$} X$ . We append an element  $e$  to an ordered list  $L$  with  $L \xleftarrow{+} e$ . To evaluate a boolean expression  $e$  to 1 or 0 we write  $\llbracket e \rrbracket$ .

<sup>8</sup> If this is done, for example, for an all-verifier key source hiding DVS (see Definition 18), one can achieve deniability against malicious verifiers.

## 2.2 Rényi entropy

The collision entropy (Rényi entropy of order  $\alpha = 2$ ) of a discrete random variable  $X$ , that can take  $n$  different values, each with probability  $p_i$  for  $i \in [n]$ , is defined as  $H_2(X) := -\log(\sum_{i=1}^n p_i^2)$ . The min-entropy (the limit of the Rényi entropy for  $\alpha \rightarrow \infty$ ) is  $H_\infty(X) := -\log \max_{i \in [n]} p_i$ .

**Fact 1.**  $H_2(X) \leq 2H_\infty(X)$ .

*Proof.*

$$H_2(X) = -\log\left(\sum_{i=1}^n p_i^2\right) \geq -\log(\max_{i \in [n]} p_i^2) = -2\log(\max_{i \in [n]} p_i) = 2H_\infty(X)$$

□

## 2.3 NP relations

For an NP-relation  $\mathcal{R}$  we define the associated language as  $\mathcal{L}_{\mathcal{R}} := \{x \mid \exists w : (x, w) \in \mathcal{R}\}$ .

**Definition 1 (Hard on average).** We say that the search problem for an NP-relation  $\mathcal{R}$  is  $(t, \varepsilon)$ -hard on average if there exists an efficient sampler  $\text{Samp}$  that outputs statements  $x \in \mathcal{L}_{\mathcal{R}}$  such that for any adversary  $\mathcal{A}$  running in time at most  $t$  we have  $\Pr[(x, w) \in \mathcal{R}] \leq \varepsilon$  where the probability is taken over  $x \xleftarrow{\$} \text{Samp}()$  and  $w \xleftarrow{\$} \mathcal{A}(x)$ .

## 2.4 NIZKs

We recall the definition of non-interactive zero knowledge (NIZK) proofs and NIZK proofs of knowledge (NIZKPoKs).

**Definition 2.** A NIZK  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  for a NP-relation  $\mathcal{R}$  with associated language  $\mathcal{L}_{\mathcal{R}} = \{x \mid \exists w : (x, w) \in \mathcal{R}\}$  consists of three algorithms:

- $\text{Gen}_{\text{NIZK}}() \xrightarrow{\$} \text{crs}$ : outputs at common reference string (crs) crs. We say that the NIZK is in the uniform random string (urs) model, if  $\text{Gen}_{\text{NIZK}}$  only outputs its random coins. In this case we call the output urs.
- $\text{Prove}(\text{crs}, x, w) \xrightarrow{\$} \pi$ : receives a crs crs and a statement witness pair  $(x, w) \in \mathcal{R}$  as input, and outputs a proof  $\pi$ .
- $\text{Vrfy}_{\text{NIZK}}(\text{crs}, x, \pi) \rightarrow \text{true/false}$ : receives a common references string crs, a statement  $x$ , and a proof  $\pi$  as input and outputs true or false, indicating a valid or invalid proof.

We require the following properties from the NIZK:

**Definition 3 (Completeness).** A NIZK  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  is  $\delta$ -complete if for all  $(x, w) \in \mathcal{R}$ ,  $\text{crs} \xleftarrow{\$} \text{Gen}_{\text{NIZK}}()$ , and  $\pi \xleftarrow{\$} \text{Prove}(\text{crs}, x, w)$  it holds that

$$\Pr[\text{Vrfy}_{\text{NIZK}}(\text{crs}, x, \pi)] \geq 1 - \delta,$$

where the probability is taken over the randomness used to sample crs and  $\pi$ .

**Definition 4 (Zero-knowledge).** A *NIZK*  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  is  $(t, t_{\text{Sim}_{\text{zk}}}, q, \varepsilon)$ -zero-knowledge if there exist algorithms

- $\text{TDGen}_{\text{zk}}() \xrightarrow{s} (\text{crs}, \text{td}_{\text{zk}})$ : outputs a crs  $\text{crs}$  and a trapdoor  $\text{td}_{\text{zk}}$  and
- $\text{Sim}_{\text{zk}}(\text{crs}, \text{td}_{\text{zk}}, x) \xrightarrow{s} \pi$ : that receives a crs  $\text{crs}$  with the corresponding trapdoor  $\text{td}_{\text{zk}}$  and a statement  $x$  as input and outputs a proof  $\pi$

with combined runtime  $t_{\text{Sim}_{\text{zk}}}$  such that for all adversaries  $\mathcal{A}$  running in time  $t$  and making at most  $q$  queries to the  $\text{PROVE}$  oracle we have

$$\text{Adv}_{\text{NIZK}}^{\text{zk}}(\mathcal{A}) := \left| \Pr[\mathcal{G}_{\text{NIZK}}^{\text{zk}}(\mathcal{A}) = 1] - \frac{1}{2} \right| \leq \varepsilon$$

where the experiment  $\mathcal{G}_{\text{NIZK}}^{\text{zk}}(\mathcal{A})$  is defined in [Figure 1](#).

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\mathcal{G}_{\text{NIZK}}^{\text{zk}}(\mathcal{A})$ : <pre> 1  <math>b \xleftarrow{s} \{0, 1\}</math> 2  <b>if</b> <math>b = 0</math> <b>then</b> 3    <math>\text{crs} \xleftarrow{s} \text{Gen}_{\text{NIZK}}()</math> 4  <b>else</b> 5    <math>(\text{crs}, \text{td}_{\text{zk}}) \xleftarrow{s} \text{TDGen}_{\text{zk}}()</math> 6  <math>b' \xleftarrow{s} \mathcal{A}^{\text{PROVE}}(\text{crs})</math> 7  <b>return</b> <math>\llbracket b' = b \rrbracket</math></pre> | $\text{PROVE}(x, w)$ : <pre> 8  <b>if</b> <math>(x, w) \notin \mathcal{R}</math> <b>then</b> 9    <b>return</b> <math>\perp</math> 10 <b>else if</b> <math>b = 0</math> 11   <b>return</b> <math>\text{Prove}(\text{crs}, x, w)</math> 12 <b>else</b> 13   <b>return</b> <math>\text{Sim}_{\text{zk}}(\text{crs}, \text{td}, x)</math></pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 1.** The zero-knowledge experiment for a *NIZK*  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$ .

**Definition 5 (Soundness).** A *NIZK*  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  is  $(t_s, \varepsilon_s)$ -sound iff for all adversaries  $\mathcal{A}$  running in time  $t_s$ , for  $\text{crs} \leftarrow \text{Gen}_{\text{NIZK}}()$  and  $(x, \pi) \xleftarrow{s} \mathcal{A}(\text{crs})$

$$\Pr[\text{Vrfy}_{\text{NIZK}}(\text{crs}, x, \pi) = 1 \wedge x \notin \mathcal{L}_{\mathcal{R}}] \leq \varepsilon_s.$$

The probability is taken over the random coins of  $\text{Gen}_{\text{NIZK}}$  and  $\mathcal{A}$ .

If a *NIZK* fulfills the following property, which is a strengthening of regular soundness, we call it a *NIZKPoK*.

**Definition 6 (Knowledge soundness).** A *NIZK*  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  is  $(t_{\text{td}}, \varepsilon_{\text{td}}, t_{\text{ks}}, \varepsilon_{\text{ks}})$ -knowledge sound iff there exists algorithms  $\text{TDGen}_{\text{ks}}$  and  $\text{Extr}$  where

- $\text{TDGen}_{\text{ks}}() \xrightarrow{s} (\text{crs}, \text{td}_{\text{ks}})$ : outputs a crs  $\text{crs}$  and an extraction trapdoor  $\text{td}_{\text{ks}}$  and
- $\text{Extr}(\text{crs}, \text{td}_{\text{ks}}, x, \text{Prove}) \xrightarrow{s} w$ : receives a crs  $\text{crs}$  with the corresponding extraction trapdoor  $\text{td}_{\text{ks}}$  and a statement  $x$  with a proof  $\text{Prove}$  as input and outputs a witness  $w$ .

$\mathcal{G}_{\text{NIZKPoK}}^{\text{pok}}(\mathcal{A})$ :

- 1  $(\text{crs}, \text{td}_{\text{ks}}) \xleftarrow{\$} \text{TDGen}_{\text{ks}}()$
- 2  $(x^*, \pi^*) \xleftarrow{\$} \mathcal{A}(\text{crs})$
- 3  $w^* \xleftarrow{\$} \text{Extr}(\text{crs}, \text{td}_{\text{ks}}, x^*, \pi^*)$
- 4 **return**  $\llbracket \text{Vrfy}_{\text{NIZK}}(\text{crs}, x, \pi) \wedge (x, w) \notin \mathcal{R} \rrbracket$

**Fig. 2.** The proof of knowledge experiment for NIZKPoK with  $\text{TDGen}_{\text{ks}}$  and  $\text{Extr}$ .

We also require for all adversaries  $\mathcal{A}$  running in time  $t_{\text{ks}}$

$$\text{Adv}_{\text{NIZKPoK}}^{\text{pok}}(\mathcal{A}) := \Pr[\mathcal{G}_{\text{NIZKPoK}}^{\text{pok}}(\mathcal{A}) = 1] \leq \varepsilon_{\text{ks}}.$$

where  $\mathcal{G}_{\text{NIZKPoK}}^{\text{pok}}(\mathcal{A})$  is defined in Figure 2, and that for all adversaries  $\mathcal{B}$  running in time  $t_{\text{td}}$  we have

$$\left| \Pr_{\text{crs} \xleftarrow{\$} \text{Gen}_{\text{NIZK}}()} [\mathcal{B}(\text{crs}) = 1] - \Pr_{(\text{crs}, \text{td}_{\text{ks}}) \xleftarrow{\$} \text{TDGen}_{\text{ks}}()} [\mathcal{B}(\text{crs}) = 1] \right| \leq \varepsilon_{\text{td}}.$$

We also recall the notion of witness indistinguishability [30], which is a relaxation of zero-knowledge. A NIZK or NIZKPoK, that does not satisfy zero-knowledge but witness indistinguishability, is called a NIWI or NIWIPoK, respectively.

**Definition 7 (Witness indistinguishable).** A NIWI  $\text{NIWI} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  for relation  $\mathcal{R}$  is  $(t, \varepsilon)$ -witness indistinguishable if for every  $\text{crs} \xleftarrow{\$} \text{Gen}_{\text{NIZK}}()$  and every adversary running in time at most  $t$  and for all  $(x, w), (x, w') \in \mathcal{R}$

$$|\Pr[\mathcal{A}(\text{crs}, \text{Prove}(\text{crs}, x, w)) \Rightarrow 1] - \Pr[\mathcal{A}(\text{crs}, \text{Prove}(\text{crs}, x, w')) \Rightarrow 1]| \leq \varepsilon$$

where the probability is taken over  $\text{crs} \xleftarrow{\$} \text{Gen}_{\text{NIZK}}()$  and the randomness of  $\mathcal{A}$  and  $\text{Prove}$ .

## 2.5 Signatures

**Definition 8 (Signature scheme).** A signature scheme  $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$  for a message space  $\mathcal{M}$  and signature space  $\mathcal{S}$  consists of three algorithms  $(\text{KeyGen}, \text{Sign}, \text{Ver})$  with the following syntax

- $\text{KeyGen}() \xrightarrow{\$} (\text{vk}, \text{sk})$ : outputs a verification key  $\text{vk}$  and a signing key  $\text{sk}$ .
- $\text{Sign}(\text{sk}, m) \xrightarrow{\$} \sigma$ : receives a signing key  $\text{sk}$  and a message  $m \in \mathcal{M}$  as input, and outputs a signature  $\sigma \in \mathcal{S}$ .
- $\text{Ver}(\text{vk}, m, \sigma) \rightarrow \text{true/false}$ : receives a verification key  $\text{vk}$ , a message  $m \in \mathcal{M}$  and a signature  $\sigma \in \mathcal{S}$  as input, and outputs true (indicating a valid signature) or false (indicating an invalid signature).

A signature scheme  $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$  is  $\delta$ -correct, if for every message  $m \in \mathcal{M}$ , we have

$$|\Pr[\text{Ver}(\text{vk}, m, \sigma) \mid (\text{vk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(), \sigma \xleftarrow{\$} \text{Sign}(\text{sk}, m)]| \geq 1 - \delta.$$

**Definition 9 (EUF-CMA).** We call a signature scheme  $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$   $(t, \varepsilon, q)$ -EUF-CMA secure, if for every adversary  $\mathcal{A}$  running in time at most  $t$  making at most  $q$  queries to the  $\text{SIGN}$  oracle we have that

$$\Pr[\mathcal{G}_{\Sigma}^{\text{euf-cma}}(\mathcal{A}) = 1] \leq \varepsilon,$$

where  $\mathcal{G}_{\Sigma}^{\text{euf-cma}}(\mathcal{A})$  is given in Figure 3.

| $\mathcal{G}_{\Sigma}^{\text{euf-cma}}(\mathcal{A})$ :                                                    | $\text{SIGN}(m)$ :                                   |
|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| 1 $Q \leftarrow \emptyset$                                                                                | 5 $\sigma \xleftarrow{\$} \text{Sign}(\text{sk}, m)$ |
| 2 $(\text{vk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}()$                                                | 6 $Q \leftarrow Q \cup \{m\}$                        |
| 3 $(m^*, \sigma^*) \xleftarrow{\$} \mathcal{A}^{\text{SIGN}}(\text{vk})$                                  | 7 <b>return</b> $\sigma$                             |
| 4 <b>return</b> $\llbracket \text{msg}^* \notin Q \wedge \text{Ver}(\text{vk}, m^*, \sigma^*) \rrbracket$ |                                                      |

**Fig. 3.** Game for EUF-CMA security of a signature scheme  $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$ .

## 2.6 Puncturable PRFs

We recall the definition of puncturable pseudorandom functions (puncturable PRFs) from [60].

**Definition 10 (Puncturable PRF).** A puncturable PRF for domain  $\mathcal{D}$  and co-domain  $\mathcal{C}$  consists of three polynomial time algorithms  $\text{PPRF} = (\text{KeyGen}, \text{Puncture}, \text{Eval})$  where

- $\text{KeyGen}() \xrightarrow{s} K$ : returns a PRF key  $K$ .
- $\text{Puncture}(K, x) \xrightarrow{s} K_x$ : takes an unpunctured PRF key  $K$  and a puncturing point  $x \in \mathcal{D}$  as input and outputs a punctured PRF key  $K_x$ .
- $\text{Eval}(K, x) \rightarrow y$ : takes a PRF key  $K$  and  $x$  as input, and outputs a value  $y \in \mathcal{C}$ .

**Definition 11 (Functionality preserving).** A puncturable PRF  $\text{PPRF}$  is  $(t_{\text{fp}}, \varepsilon_{\text{fp}})$ -functionality preserving iff for every adversary  $\mathcal{A}$  running in time at most  $t_{\text{fp}}$ ,

- for  $x \xleftarrow{\$} \mathcal{A}()$  with  $x \in \mathcal{D}$ ,
- $K \xleftarrow{\$} \text{KeyGen}()$ , and
- $K_x \xleftarrow{\$} \text{Puncture}(K, x)$ , we have

$$\Pr[\forall x' \in \mathcal{D} \setminus \{x\} : \text{Eval}(K, x') = \text{Eval}(K_x, x')] \geq 1 - \varepsilon_{\text{fp}}.$$

**Definition 12 (Pseudorandomness at punctured points).** A puncturable PRF  $\text{PPRF}$  is  $(t_{\text{pr}}, \varepsilon_{\text{pr}})$ -pseudorandom at punctured points iff for every adversary  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  running in time at most  $t_{\text{pr}}$

- for  $(x, \text{st}) \xleftarrow{\$} \mathcal{A}_1()$  with  $x \in \mathcal{D}$
- $K \xleftarrow{\$} \text{KeyGen}()$ ,
- $K_x \xleftarrow{\$} \text{Puncture}(K, x)$ , and
- $y \xleftarrow{\$} \mathcal{C}$ , we have

$$\text{Adv}_{\text{PPRF}}^{\text{pprf}}(\mathcal{A}) := |\Pr[\mathcal{A}_2(K_x, \text{Eval}(K, x), \text{st})] - \Pr[\mathcal{A}_2(K_x, y, \text{st})]| \leq \varepsilon_{\text{pr}},$$

where the probability is taken over the randomness used to sample  $K$ ,  $K_x$ ,  $y$ , and the randomness of  $\mathcal{A}_1$  and  $\mathcal{A}_2$ .

## 2.7 Indistinguishability obfuscation

We also recall the definition of indistinguishability obfuscation (iO), first conceived by [6]. We use the definition of [36].

**Definition 13 (iO).** An indistinguishability obfuscator is a PPT algorithm  $\text{iO}$  where

- $\text{iO}(C) \xrightarrow{\$} C'$ : receives as input a circuit  $C$  and outputs another circuit  $C'$ .

An indistinguishability obfuscator  $\text{iO}$  is  $\delta$ -correct iff for every circuit  $C$  with  $n$  input bits and every  $x \in \{0, 1\}^n$ , for  $C' \xleftarrow{\$} \text{iO}(C)$  we have  $\Pr[C'(x) = C(x)] \geq 1 - \delta$ .

Every indistinguishability obfuscator should satisfy the following property:

**Definition 14 (Security).** An indistinguishability obfuscator  $\text{iO}$  is  $(t_{\text{io}}, \varepsilon_{\text{io}})$ -secure iff for every adversary  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  with combined runtime at most  $t_{\text{io}}$ , where  $\mathcal{A}_1$  takes no input and outputs  $(C_0, C_1, \text{st})$  such that there exists an  $\varepsilon \in [0, 1]$  with

$$\Pr[|C_0| = |C_1| \wedge \forall x : C_0(x) = C_1(x)] \geq 1 - \varepsilon,$$

where the probability is taken over the randomness of  $\mathcal{A}_1$ , and for  $b \xleftarrow{\$} \{0, 1\}$  it holds that

$$\text{Adv}_{\text{iO}}^{\text{io}}(\mathcal{A}) := 2 \left| \Pr[\mathcal{A}_2(\text{st}, \text{iO}(C_b)) = b] - \frac{1}{2} \right| \leq \varepsilon_{\text{io}} + \varepsilon,$$

where the probability is taken over the randomness of  $b$ ,  $\mathcal{A}_1$ , and  $\mathcal{A}_2$ .

## 3 Designated verifier signatures

Designated Verifier Signature (DVS) were introduced by Jakobsson, Sako, and Impagliazzo [42]. In contrast to plain signatures, here a signature is computed for a *designated verifier*. The signature convinces only the designated verifier of the authenticity of the message but no third party. To this end, we require that



the designated verifier can *simulate* a signature. Hence, a third party cannot tell if the signer authenticated the message or if the designated verifier simulated the signature.

In this work we consider only *publicly verifiable* DVS. If the verifier's secret key is needed for verification, the scheme is called *privately verifiable* [50] or a *strong* DVS [48, 49]<sup>9</sup>.

**Definition 15 (DVS).** A designated verifier signature scheme (DVS) is a tuple of algorithms  $\text{DVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{Vrfy}, \text{Sim})$  along with a message space  $\mathcal{M}$  and identity space  $\mathcal{IDS}$ .

- $\text{Setup}() \xrightarrow{s} \text{pp}$ : A probabilistic setup algorithm that outputs public parameters.
- $\text{SKGen}(\text{pp}) \xrightarrow{s} (\text{pk}_S, \text{sk}_S)$ : A probabilistic key generation algorithm that outputs a public-/secret-key pair for the signer.
- $\text{VKGen}(\text{pp}, \text{id}_D) \xrightarrow{s} (\text{pk}_D, \text{sk}_D)$ : A probabilistic key generation algorithm that takes as input an identity and outputs a public-/secret-key pair for the verifier.
- $\text{Sign}(\text{pp}, \text{sk}_S, \text{pk}_D, \text{id}_D, m) \xrightarrow{s} \sigma / \perp$ : A probabilistic signing algorithm that takes as input a signer secret key  $\text{sk}_S$ , a designated verifier public key  $\text{pk}_D$  and an identity  $\text{id}_D \in \mathcal{IDS}$ , and a message  $m \in \mathcal{M}$ , and produces a signature  $\sigma$  or a dedicated error symbol  $\perp$ .
- $\text{Vrfy}(\text{pp}, \text{pk}_S, \text{pk}_D, \text{id}_D, m, \sigma) \rightarrow \text{true/false}$ : A deterministic verification algorithm that checks a message  $m$  and signature  $\sigma$  against a signer public key  $\text{pk}_S$  and verifier public key  $\text{pk}_D$  with identity  $\text{id}_D$ .
- $\text{Sim}(\text{pp}, \text{pk}_S, \text{sk}_D, \text{id}_D, m) \xrightarrow{s} \sigma / \perp$ : A probabilistic signature simulation algorithm that takes as input a signer public key  $\text{pk}_S$ , a designated verifier secret key  $\text{sk}_D$  and identity  $\text{id}_D \in \mathcal{IDS}$ , and a message  $m \in \mathcal{M}$ , and produces a signature  $\sigma$  or a dedicated error symbol  $\perp$ .

A DVS scheme  $\text{DVS}$  is  $\delta$ -correct, if, for any  $\text{pp} \xleftarrow{s} \text{Setup}()$ , any  $\text{id}_D \in \mathcal{IDS}$ , and honestly generated key pairs  $(\text{pk}_S, \text{sk}_S) \xleftarrow{s} \text{SKGen}(\text{pp})$ ,  $(\text{pk}_D, \text{sk}_D) \xleftarrow{s} \text{VKGen}(\text{pp}, \text{id}_D)$  and every message  $m \in \mathcal{M}$ , it holds that

$$\Pr[\text{Vrfy}(\text{pk}_S, \text{pk}_D, \text{id}_D, m, \text{Sign}(\text{sk}_S, \text{pk}_D, m))] = 1 - \delta.$$

We add the concept of identities (associated with verifier keys but not with signer keys) to achieve our novel security notions against malicious verifiers. Note that we allow several key pairs per identity. Looking ahead, in Section 3.2 we argue why verifier identities are *necessary* to achieve our novel security notions. Hence, we require separate algorithms to generate signer and verifier keys, following Laguillaumie and Vergnaud [47] and Brendel, Fiedler, Günther, Janson, and Stebila [13].

We consider two security notions: *Unforgeability* (conceptually similar to unforgeability of (standard) signature schemes) and *source hiding* (a signature does not divulge if it was created by the signer or the designated verifier).

<sup>9</sup> Other work considers a scheme *strong* if it achieves *privacy of signer's identity* [47], formalizing prose definitions of prior work [42, 59]. This property is only achievable for privately verifiable schemes.

In the following, we adapt these properties to our syntax including identities and introduce new, stronger versions allowing verifiers to generate their keys maliciously.

### 3.1 Unforgeability

Similar to unforgeability of (standard) signatures, the adversary has to create a signature for a fresh message while having access to a signing oracle. The challenger generates the sender key pair, whereas the adversary can obtain verifier public keys and verifier key pairs for identities of its choice with the VKEYS oracle, accounting for the introduction of identities. Note that we allow the adversary to decide which one of the verifier public keys to challenge, further strengthening the definition. The adversary can query for signatures designated for any honestly generated verifier public key and identity (including the lines with gray background) as previously done by [46, 13]; or, for our novel notion *malicious unforgeability*, designated for any verifier public key and identity without restriction. In particular, this models a designated verifier possibly not knowing its own secret key (“suicide attack” [42]). A similar notion has been discussed by [62, 73] for multi-designated verifier signatures under the name *unforgeability under rogue key attacks*. Prior work [38, 53, 3, 4] provides the adversary with a simulation oracle as well; we opt against it to keep a clearer distinction between unforgeability and source hiding.

Our novel malicious unforgeability is important in a setting where a signer cannot assume that verifier keys were honestly generated. Looking ahead, [Theorem 13](#) shows that signing for a maliciously generated verifier key may even leak the signer secret key if the scheme does not achieve malicious unforgeability.

**Definition 16 (Unforgeability of DVS).** *A designated verifier signature scheme DVS is  $(t, \varepsilon, q_S, q_{VK})$ -malicious unforgeable (resp. unforgeable) if, for any adversary  $\mathcal{A}$  with running time at most  $t$ , making at most  $q_S$  queries to the SIGN oracle and at most  $q_{VK}$  queries to the VKEYS oracle we have that*

$$\text{Adv}_{\text{DVS}}^{\text{mal-uf}}(\mathcal{A}) := \Pr \left[ \mathcal{G}_{\text{DVS}}^{\text{mal-uf}}(\mathcal{A}) = 1 \right] \leq \varepsilon,$$

where  $\mathcal{G}_{\text{DVS}}^{\text{mal-uf}}(\mathcal{A})$  is as defined in [Figure 4](#) and [lines with gray background](#) are considered only for (honest) unforgeability.

*Remark 1.* Note that  $(t, \varepsilon, q_S, 1)$ -malicious unforgeability implies  $(t, \varepsilon \cdot q_{VK}, q_S, q_{VK})$ -malicious unforgeability with a reduction that guesses the adversary’s output verifier key and generates the remaining verifier keys itself. For (honest) unforgeability, this reduction additionally requires the scheme to be source hiding so that the reduction can answer the SIGN queries to other designated verifiers with Sim.

```

 $\mathcal{G}_{\text{DVS}}^{\text{mal-uf}}(\mathcal{A})$ :
1   $Q_S, \mathcal{L}_V, Q_R \leftarrow \emptyset$ 
2   $\text{pp} \xleftarrow{\$} \text{Setup}()$ 
3   $(\text{pk}_S, \text{sk}_S) \xleftarrow{\$} \text{SKGen}(\text{pp})$ 
4   $(m^*, \sigma^*, \text{id}^*, \text{pk}_D^*) \xleftarrow{\$} \mathcal{A}^{\text{SIGN}, \text{VKEYS}}(\text{pp}, \text{pk}_S)$ 
5  if  $\exists \text{sk}_D^* : (\text{id}^*, \text{pk}_D^*, \text{sk}_D^*) \in \mathcal{L}_V$ 
6     $d_1 \leftarrow \text{Vrfy}(\text{pp}, \text{pk}_S, \text{pk}_D^*, \text{id}^*, m^*, \sigma^*)$ 
7     $d_2 \leftarrow (\text{id}^*, \text{pk}_D^*, m^*) \notin Q_S$ 
8     $d_3 \leftarrow (\text{id}^*, \text{pk}_D^*) \notin Q_R$ 
9    return  $\llbracket d_1 \wedge d_2 \wedge d_3 \rrbracket$ 
10 else
11   return 0

SIGN(pk, id, m):
12 if  $\neg \exists \text{sk} : (\text{id}, \text{pk}, \text{sk}) \in \mathcal{L}_V$ 
    //only accept honestly generated verifier keys
13   return  $\perp$ 
14  $Q_S \leftarrow Q_S \cup \{(\text{id}, \text{pk}, m)\}$ 
15  $\sigma \xleftarrow{\$} \text{Sign}(\text{pp}, \text{sk}_S, \text{pk}, \text{id}, m)$ 
16 return  $\sigma$ 

VKEYS(id, s):
17  $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{VKGen}(\text{pp}, \text{id})$ 
18  $\mathcal{L}_V \leftarrow \mathcal{L}_V \cup \{(\text{id}, \text{pk}, \text{sk})\}$ 
19 if  $s = \text{true}$ 
20    $Q_R \leftarrow Q_R \cup \{(\text{id}, \text{pk})\}$ 
21   return  $(\text{pk}, \text{sk})$ 
22 else
23   return  $\text{pk}$ 

```

**Fig. 4.** Unforgeability limited to honestly generated verifier keys (including the parts with gray background) or allowing malicious verifier keys (without gray background) of a designated verifier signature (cp. Definition 16).

### 3.2 Source hiding

Furthermore, we require that the DVS scheme is *source hiding*, i.e. an adversary cannot tell whether a signature was created by the sender (with  $\text{Sign}$ ) or the designated verifier (with  $\text{Sim}$ ). We follow the formalization of Brendel et al. [13], who discuss that the same property has been around with different names (designated verifier property, non-transferability, untransferability, source deniable, off-the-record). We incorporate verifier identities into the notion of [13]: The adversary chooses the verifier identity and the game honestly generates the verifier key pair and gives it to the adversary. Prior work has proposed definitions which are in some aspects weaker: The adversary does not get the secret keys [50, 49, 58, 15, 72, 65, 67] (but instead a signing oracle [15, 72]), the adversary is limited to a single challenge [50, 68, 49, 15, 72, 66, 47], or the challenge message is randomly chosen [50, 68, 49, 66].

While this provides a guarantee against a third party who can coerce users into giving up their secret keys, it does not protect against verifiers who generate their keys maliciously.

**Definition 17 (Source hiding of DVS).** *A designated verifier signature scheme DVS is  $(t, \varepsilon, q_C)$ -source hiding if, for any adversary  $\mathcal{A}$  with running time at most  $t$  and making at most  $q_C$  queries to the CHALL oracle, we have that*

$$\text{Adv}_{\text{DVS}}^{\text{sh}}(\mathcal{A}) := 2 \cdot \left| \Pr[\mathcal{G}_{\text{DVS}}^{\text{sh}}(\mathcal{A}) = 1] - \frac{1}{2} \right| \leq \varepsilon,$$

where  $\mathcal{G}_{\text{DVS}}^{\text{sh}}(\mathcal{A})$  is defined in Figure 5.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\mathcal{G}_{\text{DVS}}^{\text{sh}}(\mathcal{A})$ : <pre> 1  pp <math>\xleftarrow{\\$}</math> Setup() 2  id<sub>D</sub> <math>\leftarrow \perp</math> 3  (pk<sub>S</sub>, sk<sub>S</sub>) <math>\xleftarrow{\\$}</math> SKGen(pp) 4  b <math>\xleftarrow{\\$}</math> {0, 1} 5  b' <math>\xleftarrow{\\$}</math> <math>\mathcal{A}^{\text{CHALL}, \text{VKEY}}(\text{pp}, \text{pk}_S, \text{sk}_S)</math> 6  return <math>\llbracket b' = b \rrbracket</math> </pre>                                                                                                                                                                                                        | $\text{CHALL}(m)$ : <pre> 12 if id<sub>D</sub> = <math>\perp</math> 13   return <math>\perp</math> 14 if b = 0 15   <math>\sigma \xleftarrow{\\$}</math> Sign(pp, sk<sub>S</sub>, pk<sub>D</sub>, id<sub>D</sub>, m) 16 else 17   <math>\sigma \xleftarrow{\\$}</math> Sim(pp, pk<sub>S</sub>, sk<sub>D</sub>, id<sub>D</sub>, m) 18 return <math>\sigma</math> </pre>                                                                                                           |
| $\text{VKEY}(\text{id})$ : <pre> 7  if id<sub>D</sub> <math>\neq \perp</math>    //one-time oracle: set verifier identity at first query 8  return <math>\perp</math> 9  (pk<sub>D</sub>, sk<sub>D</sub>) <math>\xleftarrow{\\$}</math> VKGen(pp, id) 10 id<sub>D</sub> <math>\leftarrow</math> id 11 return (pk<sub>D</sub>, sk<sub>D</sub>) </pre>                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| $\mathcal{G}_{\text{DVS}, \text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux}}^{\text{all-sh}}(\mathcal{A})$ : <pre> 1  pp <math>\xleftarrow{\\$}</math> Setup() 2  (pk<sub>S</sub>, sk<sub>S</sub>) <math>\xleftarrow{\\$}</math> SKGen(pp) 3  b <math>\xleftarrow{\\$}</math> {0, 1} 4  b' <math>\xleftarrow{\\$}</math> <math>\mathcal{A}^{\text{CHALL}}(\text{pp}, \text{pk}_S, \text{sk}_S, \text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux})</math> 5  return <math>\llbracket b' = b \rrbracket</math> </pre>                                                                                                                                                           | $\text{CHALL}(m)$ : <pre> 6  if b = 0 7    <math>\sigma \xleftarrow{\\$}</math> Sign(pp, sk<sub>S</sub>, pk<sub>D</sub>, id<sub>D</sub>, m) 8  else 9    <math>\sigma \xleftarrow{\\$}</math> Sim(pp, pk<sub>S</sub>, sk<sub>D</sub>, id<sub>D</sub>, m) 10 return <math>\sigma</math> </pre>                                                                                                                                                                                    |
| $\mathcal{G}_{\text{DVS}, \text{aux}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$ : <pre> 1  pp <math>\xleftarrow{\\$}</math> Setup() 2  (pk<sub>S</sub><sup>*</sup>, sk<sub>S</sub><sup>*</sup>) <math>\xleftarrow{\\$}</math> SKGen(pp) 3  <math>\mathcal{L}_V, R \leftarrow \varepsilon</math>    //save adversary's verifier keys and randomness 4  b <math>\xleftarrow{\\$}</math> {0, 1} 5  st <math>\leftarrow \mathcal{A}^{\text{CHALL}, \text{VKEYS}, \text{RANDOM}}(\text{pp}, \text{pk}_S^*, \text{aux})</math> 6  b' <math>\xleftarrow{\\$}</math> <math>\mathcal{D}(\text{st}, \text{sk}_S^*)</math> 7  return <math>\llbracket b' = b \rrbracket</math> </pre> | $\text{CHALL}(\text{pk}, \text{id}, m)$ : //only one query <pre> 11 if (pk, id) <math>\in \mathcal{L}_V</math> //ensure malicious verifier 12   return <math>\perp</math> 13 if b = 0 14   <math>\sigma \xleftarrow{\\$}</math> Sign(pp, sk<sub>S</sub><sup>*</sup>, pk, id, m) 15 else 16   <math>\sigma \xleftarrow{\\$}</math> <math>\mathcal{E}(\text{pp}, \text{pk}_S^*, \text{aux}, R, \mathcal{L}_V, \text{pk}, \text{id}, m)</math> 17 return <math>\sigma</math> </pre> |
| $\text{VKEYS}(\text{id})$ : <pre> 8  (pk<sub>D</sub>, sk<sub>D</sub>) <math>\xleftarrow{\\$}</math> VKGen(pp, id) 9  <math>\mathcal{L}_V \xleftarrow{+}</math> (pk<sub>D</sub>, id) //append 10 return pk<sub>D</sub> </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                  | $\text{RANDOM}(n)$ : <pre> 18 r <math>\xleftarrow{\\$}</math> {0, 1}<sup>n</sup> 19 R <math>\xleftarrow{+}</math> r //append 20 return r </pre>                                                                                                                                                                                                                                                                                                                                  |

**Fig. 5.** Source hiding (top), all-verifier source hiding (middle), and simplified malicious source hiding (bottom) of a designated verifier signature (cp. [Definitions 17 to 19](#)).

We define the following strengthening of source hiding that all-quantifies over the verifier key pairs instead of generating them honestly. Looking ahead, we will use this notion as a tool for building a simplified malicious source hiding DVS in the ROM in [Section 4](#). Note that this notion is still too weak to meaningfully capture security against maliciously generated verifier keys, since it does not enforce that verifiers *know* their secret keys.

**Definition 18 (All verifier key source hiding of DVS).** *A designated verifier signature scheme DVS (with associated identity space  $\mathcal{IDS}$ ) is  $(t, \varepsilon, q_C)$ -all verifier key source hiding for the NP-relation  $\mathcal{R}$  (where  $((\text{id}, \text{pk}_D), \text{sk}_D) \in \mathcal{R}$  for all  $\text{pp} \in \text{Img}(\text{Setup}())$ ,  $\text{id} \in \mathcal{IDS}$ ,  $(\text{pk}_D, \text{sk}_D) \in \text{Img}(\text{VKGen}(\text{pp}, \text{id}))$ ) iff, for all adversaries  $\mathcal{A}$  with running time at most  $t$  that make at most  $q_C$  queries to the CHALL oracle, we have for all auxiliary input  $\text{aux} \in \{0, 1\}^*$  and for all  $((\text{id}_D, \text{pk}_D), \text{sk}_D) \in \mathcal{R}$*

$$\text{Adv}_{\text{DVS}, \text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux}}^{\text{all-sh}}(\mathcal{A}) := 2 \cdot \left| \Pr \left[ \mathcal{G}_{\text{DVS}, \text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux}}^{\text{all-sh}}(\mathcal{A}) = 1 \right] - \frac{1}{2} \right| \leq \varepsilon,$$

where  $\mathcal{G}_{\text{DVS}, \text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux}}^{\text{all-sh}}(\mathcal{A})$  is defined in [Figure 5](#).

*Remark 2.* In the definition of all verifier key source hiding, the auxiliary input can be omitted when the adversary is non-uniform, because it can have the auxiliary input hard-coded. Since we use in this work the concrete security model (and not the asymptotic security model), all adversaries are automatically non-uniform. We make the auxiliary input nevertheless explicit in our definitions, so that they can be translated easily to the asymptotic security model.

Finally, we introduce our novel *simplified malicious source hiding* definition. Intuitively, this notion models that the source of a signature remains hidden, even if the verifier generated his keys maliciously. We prefix it *simplified* since this notion alone is not yet practically meaningful, but already unachievable in the standard model as we will show in [Section 5.1](#). In the big picture, we have an adversary  $\mathcal{A}$  who can choose verifier identity and public key arbitrarily, the challenge oracle which executes **Sign** or extracts a signature from the adversary (thereby modeling that some malicious verifier could have produced a signature), and a distinguisher, who has to make a guess based on the adversary's output. This also addresses discussions of prior work [\[42, 73\]](#) of a designated verifier generating his public key in a way that he arguably does not know his own secret key and thereby undermines source hiding.

Let us go through the design choices that led to this definition. The adversary gets access to some auxiliary input  $\text{aux}$ , modeling that the adversary has access to arbitrary data generated independently of the public parameters of the scheme. Note that the auxiliary input is sampled independently of the public parameters; otherwise, the auxiliary input could include a public verifier key without the secret key or any hint how it has been generated. Simulating a signature for that key is then as hard as breaking the unforgeability of the signature. Hence, a scheme without public parameters cannot satisfy simplified malicious source

hiding. Furthermore, the public parameters for this scheme must not be reused for another scheme (without re-proving security), because this might give an adversary additional auxiliary input that depends on the public parameters.

*Design choices for identities.* Without verifier identities, an adversary can copy any verifier public key off the internet, pass it off as its own, and then argue that it could not have generated the signature itself, thereby undermining simplified malicious source hiding. To defend against this type of attack we add the concept of identities to verifier keys. A DVS scheme has to ensure that a verifier public key cannot be used with a different identity, thus precluding this attack.

*Design choices for the adversary.* Unlike for (honest) source hiding the adversary  $\mathcal{A}$  does not get the challenge signer secret key; otherwise, the extractor would get it as input as well and could trivially execute **Sign**, rendering the notion meaningless. We model the randomness of the adversary explicitly with the **RANDOM** oracle. We limit the adversary to one query to the **CHALL** oracle. We allow the adversary to obtain verifier public keys of other users via the **VKEYS** oracle. For a more realistic notion the adversary should have access to more key (pairs) and honestly generated signatures. Looking ahead, in [Section 5.1](#) we show that this notion is already unachievable in the standard model.

*Design choices for the challenge oracle.* The challenge oracle answers for verifier keys controlled by the adversary with the extractor. The extractor gets the same arguments as the adversary, and additionally all oracle responses that the adversary has obtained so far.

*Design choices for the distinguisher.* The distinguisher cannot get access to the challenge oracle since it learns the challenge signer secret key.

**Definition 19 (Simplified malicious source hiding of DVS).** *A designated verifier signature scheme DVS is  $(t_{\mathcal{A}}, t_{\mathcal{E}}, t_{\mathcal{D}}, \varepsilon, q_{VK})$ -simplified malicious source hiding iff for all adversaries  $\mathcal{A}$  with running time at most  $t_{\mathcal{A}}$  and making at most  $q_{VK}$  queries to the **VKEYS** oracle there exists an extractor  $\mathcal{E}$  running in time at most  $t_{\mathcal{E}}$  such that for any distinguisher running in time at most  $t_{\mathcal{D}}$  and for all auxiliary inputs  $\text{aux} \in \{0, 1\}^*$  we have*

$$\text{Adv}_{\text{DVS}, \text{aux}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D}) := 2 \cdot \left| \Pr[\mathcal{G}_{\text{DVS}, \text{aux}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D}) = 1] - \frac{1}{2} \right| \leq \varepsilon,$$

where  $\mathcal{G}_{\text{DVS}, \text{aux}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$  is defined in [Figure 5](#).

*Remark 3.* In the definition of simplified malicious source hiding, the auxiliary input *cannot* be omitted, even for non-uniform adversaries. The reason is that the extractor can depend in an arbitrary way on the adversary (and thus everything that is hardcoded in the adversary), but the extractor cannot depend on the auxiliary input. For example, if  $\mathcal{A}$  has a hardcoded public key, the extractor  $\mathcal{E}$  is allowed to have a corresponding secret key hardcoded (if it exists). However, if

$\mathcal{A}$  gets a public key as part of the auxiliary input, the extractor only gets access to the public key (since it gets the same auxiliary input), but no corresponding secret key.

## 4 Candidate construction

### 4.1 A DVS from NIWIPOKs

The starting point for our construction is the following simple construction of a DVS satisfying all verifier key source hiding for an arbitrary relation  $\mathcal{R}_V$ <sup>10</sup> with a  $(t, \varepsilon)$ -hard on average search problem for instance sampler **Samp** (c.f. [Definition 1](#)).

The construction makes use of an ordinary signature scheme  $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$  and a NIWIPOK NIWI for the relation

$$\mathcal{R}_N = \{((\text{id}, \text{vk}, m, x), (\sigma, w)) \mid \text{Ver}(\text{vk}, \text{id}||m, \sigma) \vee (x, w) \in \mathcal{R}_V\}$$

and is given in [Figure 6](#). The idea of using an or-proof to build a DVS first appeared in [\[42\]](#) and also appeared in [\[17\]](#) for a Multi-Designated Verifier Signature.

|                                                              |                                                                                                                              |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <u>Setup():</u>                                              | <u>Sign(crs, sk<sub>S</sub> = (vk, sk), id<sub>D</sub>, pk<sub>D</sub>, m):</u>                                              |
| 1 <b>return</b> crs $\xleftarrow{\$}$ Gen <sub>NIWI</sub> () | 5 $\sigma \xleftarrow{\$}$ Sign(sk, id <sub>D</sub>   m)                                                                     |
| <u>SKGen(crs):</u>                                           | 6 <b>return</b> $\pi \xleftarrow{\$}$ Prove(crs, (id <sub>D</sub> , vk, m, pk <sub>D</sub> ), ( $\sigma$ , $\perp$ ))        |
| 2 (vk, sk) $\xleftarrow{\$}$ KeyGen()                        | <u>Sim(pp, pk<sub>S</sub>, sk<sub>D</sub>, id<sub>D</sub>, m):</u>                                                           |
| 3 <b>return</b> (vk, (vk, sk))                               | 7 <b>return</b> $\pi \xleftarrow{\$}$ Prove(crs, (id <sub>D</sub> , vk, m, pk <sub>D</sub> ), ( $\perp$ , sk <sub>D</sub> )) |
| <u>VKGen(crs, id<sub>D</sub>):</u>                           | <u>Vrfy(crs, pk<sub>S</sub>, id<sub>D</sub>, pk<sub>D</sub>, m, <math>\pi</math>):</u>                                       |
| 4 <b>return</b> (x, w) $\xleftarrow{\$}$ Samp()              | 8 <b>return</b> Vrfy <sub>NIWI</sub> (crs, (id <sub>D</sub> , pk <sub>S</sub> , m, pk <sub>D</sub> ), $\pi$ )                |

**Fig. 6.** Our construction of a DVS  $\text{DVS}[\text{Samp}, \Sigma, \text{NIWI}] = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{Sim}, \text{Vrfy})$  that satisfies all verifier key source hiding built from an instance sampler **Samp** for an NP relation, an ordinary signature scheme  $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$ , and a NIWIPOK  $\text{NIWI} = (\text{Gen}_{\text{NIWI}}, \text{Prove}, \text{Vrfy}_{\text{NIWI}})$ .

**Theorem 2 (Correctness).** *If  $\Sigma$  is  $\delta_1$ -correct and NIWI is  $\delta_2$ -correct, then  $\text{DVS}[\text{Samp}, \Sigma, \text{NIWI}]$  as given in [Figure 6](#) is  $\delta_1 + \delta_2$ -correct.*

*Proof.* The signature  $\sigma \xleftarrow{\$} \text{Sign}(\text{sk}, \text{id}_D||m)$  verifies with probability at least  $1 - \delta_1$  and in this case  $(\sigma, \perp)$  is a valid witness for the statement  $(\text{id}_D, \text{vk}, m, \text{pk}_D)$  in  $\mathcal{R}_N$ . Since NIWI is  $\delta_2$ -correct, this results in  $\text{DVS}[\text{Samp}, \Sigma, \text{NIWI}]$  being  $\delta_1 + \delta_2$ -correct.  $\square$

<sup>10</sup> Since we are aiming for all verifier key source hiding we can ignore identities for this scheme.

**Theorem 3 (Malicious unforgeability).** *If  $\Sigma$  is  $(t, q_S, \varepsilon_\Sigma)$ -EUF-CMA secure, the search problem with instance sampler  $\text{Samp}$  for  $\mathcal{R}_V$  is  $(t, \varepsilon_{\mathcal{R}_V})$ -hard on average, and NIWI is  $(t, \varepsilon_{\text{td}}, t, \varepsilon_{\text{ks}})$ -knowledge sound with algorithms  $\text{TDGen}_{\text{ks}}$  and  $\text{Extr}$ , then  $\text{DVS}[\text{Samp}, \Sigma, \text{NIWI}]$  as given in Figure 6 is  $(t', \varepsilon, q_S, q_{VK})$ -malicious unforgeable for arbitrary  $q_{VK}$ ,  $t' \approx t$ , and  $\varepsilon \leq \varepsilon_{\text{td}} + \varepsilon_{\text{ks}} + \varepsilon_\Sigma + \varepsilon_{\mathcal{R}_V} \cdot q_{VK}$ .*

*Proof.* The proof uses a hybrid argument. Let  $G_0$  be the real malicious unforgeability game for  $\text{DVS}[\text{Samp}, \Sigma, \text{NIWI}]$  and let  $G_1$  be the same game, except that the crs for NIWI is sampled as  $(\text{crs}, \text{td}_{\text{ks}}) \xleftarrow{\$} \text{TDGen}_{\text{ks}}()$  instead of using  $\text{crs} \xleftarrow{\$} \text{Gen}_{\text{NIWI}}()$ .

**Lemma 1** ( $G_0 \rightsquigarrow G_1$ ).  $|\Pr[G_0^A \Rightarrow 1] - \Pr[G_1^A \Rightarrow 1]| \leq \varepsilon_{\text{td}}$

The proof is a straightforward reduction to the first condition of knowledge soundness.

**Lemma 2** ( $G_1$ ).  $\Pr[G_1^A \Rightarrow 1] \leq \varepsilon_{\text{ks}} + \varepsilon_\Sigma + \varepsilon_{\mathcal{R}_V} \cdot q_{VK}$

*Proof.* We give a reduction to the EUF-CMA security of  $\Sigma$ . The reduction generates  $(\text{crs}, \text{td}_{\text{ks}}) \xleftarrow{\$} \text{TDGen}_{\text{ks}}()$  and sends  $\text{crs}$  along with the public key  $\text{pk}_S$  obtained from the signature scheme to the verifier. The reduction simulates all  $\text{VKEYS}(\text{id}, s)$  queries as in the real game. It simulates  $\text{SIGN}(\text{pk}, \text{id}, m)$  queries by querying  $\text{SIGN}(\text{id}||m)$  for the underlying signature scheme to get  $\sigma$ , computing  $\pi \xleftarrow{\$} \text{Prove}(\text{crs}, (\text{id}_D, \text{vk}, m, \text{pk}_D), (\sigma, \perp))$  and sending  $\pi$  to the adversary. We omit  $\text{VRFY}$  oracle queries here, because they are unnecessary for a DVS, since the adversary can verify the signatures itself. This simulates the view of the adversary in the malicious unforgeability game perfectly.

When the adversary outputs  $(m^*, \pi^*, \text{id}^*)$  where  $\text{id}^*$  has been queried to  $\text{VKEYS}$  before and answered with  $\text{pk}_D^*$  and  $\text{Vrfy}_{\text{NIWI}}(\text{crs}, (\text{id}^*, \text{pk}_S, m^*, \text{pk}_D^*), \pi^*) = \text{true}$ , the reduction runs  $(\sigma, w) \xleftarrow{\$} \text{Extr}(\text{crs}, \text{td}_{\text{ks}}, (\text{id}^*, \text{pk}_S, m^*, \text{pk}_D^*), \pi^*)$  and submits  $\sigma$  as signature for  $\text{id}^*||m^*$  for the underlying signature scheme. When  $(\text{id}^*, m^*) \notin Q$ , the reduction did not query for a signature for  $\text{id}^*||m^*$  before.

The signature produced by the reduction is guaranteed to verify if the adversary was successful and none of the following two bad events occurred:

- $\text{Bad}_1 := (\sigma, w) \notin \mathcal{R}_N$
- $\text{Bad}_2 := (\sigma, w) \in \mathcal{R}_N \wedge \text{Ver}(\text{pk}_S, \text{id}^*||m^*, \sigma) = \text{false}$

We next bound the probability of these two bad events.

*Claim 4.*  $\Pr[\text{Bad}_1] \leq \varepsilon_{\text{ks}}$

*Proof.* We describe a simple reduction that wins the second part of the knowledge soundness game if  $\text{Bad}_1$  occurs. The reduction receives  $\text{crs}$  and simulates all other parts of the malicious unforgeability game itself. When the adversary outputs  $(m^*, \pi^*, \text{id}^*)$ , it submits  $((\text{id}^*, \text{pk}_S, m^*, \text{pk}_D^*), \pi^*)$  (where  $\text{pk}_S$  is the signer public key generate initially and  $\text{pk}_D^*$  is the verifier public key generated for  $\text{id}^*$ ) to the unforgeability game. Clearly, the adversary's view is exactly as in  $G_1$  and the reduction wins the knowledge soundness game if  $\text{Bad}_1$  occurs.  $\square$

*Claim 5.*  $\Pr[\text{Bad}_2] \leq \varepsilon_{\mathcal{R}_V} \cdot q_{VK}$



*Proof.* We give a simple reduction that solves the search problem for  $\mathcal{R}_V$  on average, if  $\text{Bad}_2$  occurs. The reduction is given a random instance  $x$  and simulates the game  $G_1$  for the adversary honestly, except that it answers the  $i$ -th query to the VKEYS oracle (where  $i \leftarrow^{\$} [q_{VK}]$ ) with  $x$  as public key.

When the adversary outputs  $(m^*, \pi^*, \text{id}^*)$  and  $\text{id}^*$  has been queried in the  $i$ -th VKEYS query before (which happens with probability  $1/q_{VK}$ ) and  $\text{Vrfy}_{\text{NIWI}}(\text{crs}, (\text{id}^*, \text{pk}_S, m^*, x), \pi^*) = \text{true}$ , the reduction runs  $(\sigma, w) \leftarrow^{\$} \text{Extr}(\text{crs}, \text{td}_{\text{KS}}, (\text{id}^*, \text{pk}_S, m^*, x), \pi^*)$  and sends  $w$  to its game.

Clearly, the adversary's view is exactly as in  $G_1$  and the reduction solves the search problem game if  $\text{Bad}_2$  occurs, since  $\text{Bad}_2$  implies  $(x, w) \in \mathcal{R}_V$ .  $\square$

Combining [Claims 4](#) and [5](#) with the analysis of the reduction to the EUF-CMA security of  $\Sigma$  yields [Lemma 2](#).  $\square$

[Theorem 3](#) follows from combining [Lemmata 1](#) and [2](#).  $\square$

**Theorem 6 (All verifier key source hiding).** *If NIWI is  $(t, \varepsilon)$ -witness indistinguishable, then  $\text{DVS}[\text{Samp}, \Sigma, \text{NIWI}]$  as given in [Figure 6](#) is  $(t', \varepsilon', q_C)$ -all verifier key source hiding for arbitrary  $q_C$ ,  $t' \approx t$ , and  $\varepsilon' \leq q_C \cdot \varepsilon$ .*

*Proof.* We describe a reduction for one CHALL query. The argument can be extended to many CHALL queries with a standard hybrid argument.

The reduction has the game parameters  $\text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux}$  hardcoded. It gets  $\text{crs}$  from the witness indistinguishability game and samples  $(\text{pk}_S, \text{sk}_S) \leftarrow^{\$} \text{SKGen}(\text{crs})$  and runs the adversary on  $\text{pp}, \text{pk}_S, \text{sk}_S, \text{id}_D, \text{pk}_D, \text{sk}_D, \text{aux}$ . When the adversary makes a challenge query on message  $m$ , the reduction answers this with a proof for the statement  $(\text{id}_D, \text{vk}, m, \text{pk}_D)$  generated via the witness indistinguishability game with witnesses  $(\sigma, \perp)$  and  $(\perp, \text{sk}_D)$ , where  $\sigma \leftarrow^{\$} \text{Sign}(\text{sk}, \text{id}_D || m)$ . The reduction outputs the same bit that the adversary outputs.

If the witness indistinguishability game uses the witness  $(\sigma, \perp)$ , the reduction perfectly simulates the all verifier key source hiding game for  $b = 0$ . If the witness  $(\perp, \text{sk}_D)$  is used, the reduction perfectly simulates the game for  $b = 1$ .

Hence, we get  $\varepsilon' \leq \varepsilon$  for a single CHALL query, or  $\varepsilon' \leq q_C \cdot \varepsilon$  for multiple challenge queries with a hybrid argument.  $\square$

## 4.2 Construction in the random oracle model.

In the random oracle model each algorithm (including the adversary and the extractor) has oracle access to a uniformly random function  $f : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$  that models a cryptographic hash function. For our definitions with auxiliary input, we allow the auxiliary input to depend on  $q_{\mathcal{H}, \text{aux}}$  function values of  $f$ .

In this section we present a generic transformation of an all verifier key source hiding DVS  $\text{DVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{Vrfy})$  and a NIZKPoK in the uniform random string (urs) model for the relation

$$\mathcal{R} = \{((\text{pp}, \text{id}_D, \text{pk}_D), r) \mid \exists \text{sk}_D : (\text{pk}_D, \text{sk}_D) \leftarrow \text{VKGen}(\text{pp}, \text{id}_D; r)\}$$

to a simplified malicious source hiding DVS in the random oracle model (ROM). The result is in contrast to our impossibility result presented in [Section 5](#) which

works for any construction in the standard model. This shows that replacing the random oracle with any concrete hash functions (which has to be done to deploy such a scheme in the real world, where a random oracle is not available) yields an insecure construction.

If the underlying DVS is unforgeable or malicious unforgeable, then the resulting DVS achieves the same notion of unforgeability. The construction is given in Figure 7.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>Setup'():</u></p> <ol style="list-style-type: none"> <li>1 <math>\text{pp} \xleftarrow{\\$} \text{Setup}()</math></li> <li>2 <math>s \xleftarrow{\\$} \{0, 1\}^\lambda</math></li> <li>3 <b>return</b> <math>\text{pp}' = (\text{pp}, s)</math></li> </ol> <p><u>SKGen'(pp' = (pp, s)):</u></p> <ol style="list-style-type: none"> <li>4 <b>return</b> SKGen(pp)</li> </ol> <p><u>VKGen'(pp' = (pp, s), id<sub>D</sub>):</u></p> <ol style="list-style-type: none"> <li>5 <math>(\text{pk}_D, \text{sk}_D) \leftarrow \text{VKGen}(\text{pp}, \text{id}_D; r)</math></li> <li>6 <math>\text{urs} \leftarrow \mathcal{H}(s, \text{id}_D, \text{pk}_D)</math></li> <li>7 <math>\pi \xleftarrow{\\$} \text{Prove}(\text{urs}, (\text{pp}, \text{id}_D, \text{pk}_D), r)</math></li> <li>8 <b>return</b> <math>(\text{pk}'_D = (\text{pk}_D, \pi), (\text{pk}_D, \text{sk}_D))</math></li> </ol> <p><u>Vrfy'(pp, pk<sub>S</sub>, pk'<sub>D</sub> = (pk<sub>D</sub>, π), id<sub>D</sub>, m, σ):</u></p> <ol style="list-style-type: none"> <li>9 <b>return</b> Vrfy(pp, pk<sub>S</sub>, pk<sub>D</sub>, id<sub>D</sub>, m, σ)</li> </ol> | <p><u>Sign'(pp' = (pp, s), sk<sub>S</sub>, id<sub>D</sub>, pk'<sub>D</sub>, m):</u></p> <ol style="list-style-type: none"> <li>10 <math>(\text{pk}_D, \pi) \leftarrow \text{pk}'_D</math></li> <li>11 <math>\text{urs} \leftarrow \mathcal{H}(s, \text{id}_D, \text{pk}_D)</math></li> <li>12 <b>if</b> Vrfy<sub>NIZK</sub>(urs, (pp, id<sub>D</sub>, pk<sub>D</sub>), π)</li> <li>13     <b>return</b> Sign(pp, sk<sub>S</sub>, id<sub>D</sub>, pk<sub>D</sub>, m)</li> <li>14 <b>else</b></li> <li>15     <b>return</b> ⊥</li> </ol> <p><u>Sim'(pp, pk<sub>S</sub>, sk'<sub>D</sub> = (pk<sub>D</sub>, sk<sub>D</sub>), id<sub>D</sub>, m):</u></p> <ol style="list-style-type: none"> <li>16 <math>\text{urs} \leftarrow \mathcal{H}(s, \text{id}_D, \text{pk}_D)</math></li> <li>17 <b>if</b> Vrfy<sub>NIZK</sub>(urs, (pp, id<sub>D</sub>, pk<sub>D</sub>), π)</li> <li>18     <b>return</b> Sim(pp, pk<sub>S</sub>, sk<sub>D</sub>, id<sub>D</sub>, m)</li> <li>19 <b>else</b></li> <li>20     <b>return</b> ⊥</li> </ol> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 7.** Our construction of a simplified malicious source hiding DVS  $\text{DVS}'[\text{DVS}, \text{NIZK}] = (\text{Setup}', \text{SKGen}', \text{VKGen}', \text{Sign}', \text{Sim}', \text{Vrfy}')$  built from an all verifier key source hiding DVS  $\text{DVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{Sim}, \text{Vrfy})$ , a NIZKPOK  $\text{NIZK} = (\text{Gen}_{\text{NIZK}}, \text{Prove}, \text{Vrfy}_{\text{NIZK}})$  in the ur model with a uniform reference string of length  $\ell$  and a hash function  $\mathcal{H} : \{0, 1\}^\lambda \times \mathcal{IDS} \times \mathcal{VPKS} \rightarrow \{0, 1\}^\ell$ , where  $\mathcal{IDS}$  is the identity space and  $\mathcal{VPKS}$  the public key space of the verifiers, modeled as a random oracle.

**Theorem 7 (Correctness).** *If DVS is  $\delta$ -correct and NIZK is  $\delta'$ -correct, then  $\text{DVS}'[\text{DVS}, \text{NIZK}]$  as given in Figure 7 is  $(\delta + \delta')$ -correct.*

*Proof.* Let  $\text{id}_D \in \mathcal{IDS}$  and  $\text{pp} \xleftarrow{\$} \text{Setup}()$ ,  $(\text{pk}_S, \text{sk}_S) \xleftarrow{\$} \text{SKGen}(\text{pp})$ ,  $(\text{pk}_D, \text{sk}_D) \leftarrow \text{VKGen}(\text{pp}, \text{id}_D; r)$ , and  $\pi \xleftarrow{\$} \text{Prove}(\text{urs}, (\text{pp}, \text{id}_D, \text{pk}_D), r)$ . Since NIZK is  $\delta'$ -correct, with probability  $1 - \delta'$  we have  $\text{Vrfy}_{\text{NIZK}}(\text{urs}, (\text{pp}, \text{id}_D, \text{pk}_D), \pi) = \text{true}$  and in this case  $\text{Sign}'(\text{pp}, \text{sk}_S, \text{id}_D, \text{pk}'_D = (\text{pk}_D, \pi), m)$  returns  $\sigma \xleftarrow{\$} \text{Sign}(\text{pp}, \text{sk}_S, \text{id}_D, \text{pk}_D, m)$ . Since DVS is  $\delta$ -correct,  $\text{Vrfy}'(\text{pp}, \text{pk}_S, \text{pk}'_D = (\text{pk}_D, \pi), m, \sigma) = \text{Vrfy}(\text{pp}, \text{pk}_S, \text{pk}_D, m, \sigma) = \text{true}$  with probability  $1 - \delta$ .  $\square$

**Theorem 8 ((Malicious) unforgeability).** *If DVS is  $\delta$ -correct,  $(t, \varepsilon, q_S, q_{VK})$ -(malicious) unforgeable with  $\varepsilon < 1/2$ , and  $(t, \varepsilon_{\text{sh}}, 1)$ -source hiding with  $4\varepsilon_{\text{sh}} + 2\delta \leq 1 - \varepsilon$  and NIZK is  $(t_{\text{zk}}, t_{\text{Sim}_{\text{zk}}}, \varepsilon_{\text{zk}})$ -zero-knowledge, then  $\text{DVS}'[\text{DVS}, \text{NIZK}]$  as given in Figure 7 is  $(t', \varepsilon', q_S, q_{VK})$ -(malicious) unforgeable against adversaries making at most  $q_{\mathcal{H}}$  ROM queries with  $t' \approx t$  and  $\varepsilon' \leq \varepsilon + q_{VK}\varepsilon_{\text{zk}} + q_{\mathcal{H}}\sqrt{2\varepsilon} + 4\varepsilon_{\text{sh}} + 2\delta$ .*

*Proof.* Assume there is an adversary  $\mathcal{A}$  that breaks the  $(t', \varepsilon', q_S, q_V, q_{VK})$ -(malicious) unforgeability of  $\text{DVS}'[\text{DVS}, \text{NIZK}]$ . Let  $\text{TDGen}_{\text{zk}}$  and  $\text{Extr}$  be the algorithms for the zero-knowledge property of  $\text{NIZK}$ . The proof proceeds via a hybrid argument. Let  $G_0$  be the real (malicious) unforgeability game for  $\text{DVS}'[\text{DVS}, \text{NIZK}]$  and  $G_1$  the same game with the following modification: If the adversary makes a  $\text{VKEYS}$  query on  $\text{id}$  and that query is answered with  $(\text{pk}, \pi)$  and the adversary has made a  $\text{ROM}$  query on  $(s, \text{id}, \text{pk})$  before, the game aborts and outputs a random bit.

**Lemma 3** ( $G_0 \rightsquigarrow G_1$ ).  $|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq q_{\mathcal{H}} \sqrt{2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta}$

*Proof.* Consider the reduction  $\mathcal{B}$  against the (malicious) unforgeability of  $\text{DVS}$  that proceeds as follows: The adversary gets  $(\text{pp}, \text{pk}_S)$  and makes  $\text{VKEYS}$  query on identities, that are chosen exactly how  $\mathcal{A}$  chooses the identities for its  $\text{VKEYS}$  queries, to get for each query a public key  $\text{pk}$  and then sample  $(\text{pk}', \text{sk}') \xleftarrow{\$} \text{VKGen}(\text{pp}, \text{id})$ . If  $\text{pk}' \neq \text{pk}$  for all queries, the adversary aborts. Otherwise, it computes a signature via  $\sigma \xleftarrow{\$} \text{Sim}(\text{pp}, \text{pk}_S, \text{sk}', \text{id}, m)$  for any fixed message  $m$ . The reduction then outputs  $(m, \sigma, \text{id}_D, \text{pk})$ .

We will show next that  $\Pr[\text{Vrfy}(\text{pp}, \text{pk}_S, \text{pk}, \text{id}, m, \sigma) = \text{true}] \geq 1/2 - 2\varepsilon_{\text{sh}} - \delta$ . Consider for this the reduction  $\mathcal{B}'$  that plays the source hiding game for  $\text{DVS}$  as follows:  $\mathcal{B}'$  receives  $\text{pp}, \text{pk}_S$  (and  $\text{sk}_S$  which is ignored by  $\mathcal{B}'$ ) and makes one  $\text{VKEYS}$  query on  $\text{id}$  to obtain  $\text{pk}_D$  (and  $\text{sk}_D$ , which is again ignored) and then a  $\text{CHALL}$  query on  $m$  to obtain  $\sigma$ . The inputs  $\text{id}$  and  $m$  are chosen exactly how  $\mathcal{B}$  chooses them.  $\mathcal{B}'$  outputs  $b' = 0$  if  $\text{Vrfy}(\text{pp}, \text{pk}_S, \text{pk}_D, \text{id}, m, \sigma) = \text{true}$  and  $b' = 1$  otherwise. The reduction  $\mathcal{B}'$  only fails when  $\text{Sign}$  produced a signature that does not verify or if  $\text{Sim}$  produced a signature that did verify. The first event occurs with probability at most  $\delta$ . Let  $E$  be the second event, i.e., the signature computed by  $\text{Sim}$  verifies. Then  $1 - \delta - \Pr[E] \leq 1/2 + 2\varepsilon_{\text{sh}}$ . Rearranging gives the desired lower bound on  $\Pr[E]$ .

This shows that the reduction  $\mathcal{B}$  wins the (malicious) unforgeability game with probability  $\Pr[\text{pk} = \text{pk}'] \cdot (1/2 - 2\varepsilon_{\text{sh}} - \delta) \leq \varepsilon$  which shows  $\Pr[\text{pk} = \text{pk}'] \leq \varepsilon / (1/2 - 2\varepsilon_{\text{sh}} - \delta) = 2\varepsilon / (1 - 4\varepsilon_{\text{sh}} - 2\delta) \leq 2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta$ . See [Lemma 11](#) in [Appendix C](#) for an explanation of the last inequality.

Thus we have  $2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta \geq \Pr[\text{pk} = \text{pk}'] = 2^{-H_2(\text{pk})}$  which can be rearranged to  $H_2(\text{pk}) \geq -\log(2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta)$ . Thus we also have  $H_{\infty}(\text{pk}) \geq \frac{-1}{2} \log(2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta)$  by [Fact 1](#), which implies that any public key  $\text{pk}'$  used by  $\mathcal{A}$  in a  $\text{ROM}$  query matches a public key  $\text{pk}$  generated in a  $\text{VKEYS}$  oracle query at most with probability  $2^{-H_{\infty}(\text{pk})} \leq 2^{\frac{1}{2} \log(2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta)} = \sqrt{2\varepsilon + 4\varepsilon_{\text{sh}} + 2\delta}$ .  $\square$

The next game,  $G_2$ , is defined like  $G_1$ , except that all  $\text{VKEYS}(\text{id})$  queries are now answered by generating the  $\text{urs}$  with known trapdoor and simulating the random oracle accordingly. In particular, by

- sampling  $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{VKGen}(\text{pp}, \text{id})$ , programming the random oracle query on  $(s, \text{id}, \text{pk})$  with  $\text{urs}$  sampled via  $(\text{urs}, \text{td}_{\text{zk}}) \xleftarrow{\$} \text{TDGen}_{\text{zk}}()$  (or, as in  $G_1$ , aborting if the adversary already queried the random oracle on  $(s, \text{id}, \text{pk})$ ),
- sampling  $\pi \xleftarrow{\$} \text{Sim}_{\text{zk}}(\text{urs}, \text{td}_{\text{zk}}, (\text{pp}, \text{id}_D, \text{pk}_D))$ , and
- returning  $((\text{pk}, \pi), \text{sk})$ .

**Lemma 4** ( $G_1 \rightsquigarrow G_2$ ).  $|\Pr[G_1^A \Rightarrow 1] - \Pr[G_2^A \Rightarrow 1]| \leq q_{VK}\varepsilon_{zk}$

*Proof.* We give a reduction to the zero-knowledge property of NIZK for the  $i$ -th query. The transition from  $G_1$  to  $G_2$  follows then from a standard hybrid argument. The reduction samples  $\mathbf{pp}' = (\mathbf{pp}, s)$  and  $(\mathbf{pk}_S, \mathbf{sk}_S) \xleftarrow{\$} \text{SKGen}(\mathbf{pp})$  and sends them to  $\mathcal{A}$ . When the adversary makes its  $i$ -th VKEYS query on  $\text{id}$ , the reduction samples  $(\mathbf{pk}, \mathbf{sk}) \xleftarrow{\$} \text{VKGen}(\mathbf{pp}, \text{id})$ . If the adversary has already made a random oracle query on  $(s, \text{id}, \mathbf{pk})$ , the reduction aborts and outputs a random bit. Otherwise, the reduction uses  $\mathbf{urs}$  from the zero-knowledge game (i.e., its original input) and programs the random oracle to return  $\mathbf{urs}$  on  $(s, \text{id}, \mathbf{pk})$ . The reduction queries the PROVE oracle on  $((\mathbf{pp}, \text{id}, \mathbf{pk}), \mathbf{sk})$  to retrieve  $\pi$  and sends  $(\mathbf{pk}, \pi)$  to the adversary. The reduction answers SIGN and VRFY queries as in  $G_1$ .

When the reduction does not abort, the reduction simulates the game  $G_1$  if the zero-knowledge game for NIZK is played with  $b = 0$  and otherwise the reduction simulates  $G_2$ . If the reduction does abort, the game  $G_1$  and  $G_2$  both abort, too, and thus the reduction has simulated them correctly.  $\square$

**Lemma 5** ( $G_2$ ).  $\Pr[G_2^A \Rightarrow 1] \leq \varepsilon$

*Proof.* For this proof we assume without loss of generality that  $\mathcal{A}$  queries the ROM on  $(s, \text{id}, \mathbf{pk})$  before making a sign query on  $\text{SIGN}(\mathbf{pk}, \text{id}, \cdot)$ .

We give a reduction to the (malicious) unforgeability of DVS. Initially, the reduction receives  $(\mathbf{pp}, \mathbf{pk}_S)$ , samples  $s \xleftarrow{\$} \{0, 1\}^\lambda$  and sends  $(\mathbf{pp}', s) := (\mathbf{pp}, s), \mathbf{pk}_S$  to the adversary. If the adversary makes a VKEYS( $\text{id}$ ) query, the reduction forwards the query to obtain  $\mathbf{pk}$ , aborts if the adversary already made a ROM query on  $(s, \text{id}, \mathbf{pk})$  and otherwise programs this query to  $\mathbf{urs}$  sampled via  $(\mathbf{urs}, \mathbf{td}_{zk}) \xleftarrow{\$} \text{TDGen}_{zk}()$  and computes  $\pi \xleftarrow{\$} \text{Sim}_{zk}(\mathbf{urs}, \mathbf{td}_{zk}, (\mathbf{pp}, \text{id}_D, \mathbf{pk}_D))$ . The reduction sends the answer  $(\mathbf{pk}, \pi)$  to the adversary.

The reduction perfectly simulates the game  $G_2$  for the adversary and thus wins the (malicious) unforgeability game for DVS exactly when the adversary wins  $G_2$ .  $\square$

Combining Lemma 3 – Lemma 5 yields Theorem 8.  $\square$

*Remark 4.* The argument in Lemma 3 for the min-entropy in the verifier public keys makes our reduction quite loose. However, we can obtain a tight reduction by appending random bits to the verifier public keys to increase their min-entropy.

The simplified malicious source hiding proof makes use of the following Lemma, which is a special case of [37, Lemma 6], which in turn relies on the “Bucket Lemma” by [43].

Let  $\mathcal{D}$  be a distribution on a set  $\mathcal{X}$  and  $g : \mathcal{X} \rightarrow \mathcal{Y}$  and  $h : \mathcal{X} \rightarrow \mathcal{Z}$  functions where  $\mathcal{Y}$  is a finite set and  $\mathcal{Z}$  is any set. Moreover, for  $X \in \mathcal{X}$  let  $\mathcal{D} \mid [h(\cdot) = h(X)]$  be the distribution that samples  $X' \xleftarrow{\$} \mathcal{D}$  conditioned on  $h(X') = h(X)$ .

**Lemma 6 (Split resampling Lemma [37, Lemma 6]).** *Let  $T^{\text{rep}}$  be the number of iterations of the loop in lines 2–4 of algorithm  $\mathcal{A}()$  given in Figure 8. Then for all  $\gamma \in (0, 1]$  we have*

$$\Pr\left[T^{\text{rep}} \leq 2 \ln\left(\frac{2}{\gamma}\right) \frac{|\mathcal{Y}|}{\gamma}\right] \geq 1 - \gamma.$$

```

 $\mathcal{A}()$ :
1  $X \xleftarrow{\$} \mathcal{D}$ 
2 repeat
3    $X' \xleftarrow{\$} \mathcal{D} \mid [h(\cdot) = h(X)]$ 
4 until  $g(X') = g(X)$ 
5 return  $X'$ 

```

Fig. 8. Algorithm for the split resampling Lemma.

**Theorem 9 (Simplified malicious source hiding).** *If DVS is  $(t, \varepsilon_{\text{sh}}, 1)$ -all verifier key source hiding and NIZK is  $(t_{\text{td}}, \varepsilon_{\text{td}}, t_{\text{ks}}, \varepsilon_{\text{ks}})$ -knowledge sound, then  $\text{DVS}'[\text{DVS}, \text{NIZK}]$  as given in Figure 7 is  $(t_{\mathcal{A}}, t_{\mathcal{E}}, t_{\mathcal{D}}, \varepsilon, q_{\text{VK}})$ -simplified malicious source hiding with  $t_{\mathcal{E}} \approx 2 \ln(2/\gamma)(q_{\mathcal{H}} + 1)/\gamma \cdot t_{\mathcal{A}}$ ,  $t_{\text{td}} \approx t_{\text{ks}} \approx t \approx t_{\mathcal{A}} + t_{\mathcal{D}}$ , and  $\varepsilon \leq \varepsilon_{\text{ks}} + 2 \ln(2/\gamma)(q_{\mathcal{H}} + 1)\varepsilon_{\text{td}} + q_{\mathcal{H}, \text{aux}}/2^\lambda$ , where  $q_{\mathcal{H}, \text{aux}}$  is the number of random oracle queries that  $\text{aux}$  depends on. The parameters  $\lambda \in \mathbb{N}$  and  $\gamma \in (0, 1]$  can be selected arbitrarily to achieve the desired security properties.*

*Proof.* Let  $\mathcal{A}$  be an adversary against the simplified malicious source hiding property of  $\text{DVS}'[\text{DVS}, \text{NIZK}]$ . We first observe that  $\text{aux}$  does not depend on any random oracle query that starts with  $s$ , where  $s$  is part of the public parameters, except with probability  $q_{\mathcal{H}, \text{aux}}/2^\lambda$ , because  $s$  is a uniformly random  $\lambda$  bit string generated independently of  $\text{aux}$ . We assume in the following that  $\text{aux}$  does not depend on such a query.

We next describe the extractor  $\mathcal{E}$ . The extractor is given  $\text{pp}' = (\text{pp}, s), \text{pk}_S^*, \text{aux}, R, \mathcal{L}_V, \text{pk}' = (\text{pk}, \pi), \text{id}, m$ , where  $R$  are the random coins consumed by the adversary so far. If  $\text{Vrfy}_{\text{NIZK}}(\text{urs}, (\text{pp}, \text{id}, \text{pk}), \pi) = \text{false}$  for  $\text{urs} \leftarrow \mathcal{H}(s, \text{id}, \text{pk})$ , the extractor outputs  $\perp$ .

Otherwise, the extractor runs  $\mathcal{A}$  several times, while changing some query answers. We assume without loss of generality that  $\mathcal{A}$  queries the RO for each value at most once and makes a query on  $(s, \widetilde{\text{id}}, \widetilde{\text{pk}})$  before the  $\text{CHALL}(\widetilde{\text{pk}}' = (\widetilde{\text{pk}}, \widetilde{\pi}), \widetilde{\text{id}}, \widetilde{m})$  query. We also assume without loss of generality that the number of random coins consumed and the number of random oracle queries by  $\mathcal{A}$  until a  $\text{CHALL}$  query are fixed values.

The extractor runs  $\mathcal{A}$  using the random coins  $R$  and answers all queries as described before, except when  $\mathcal{A}$  makes the random oracle query on  $(s, \text{id}, \text{pk})$ . This ROM query is answered by sampling  $(\text{urs}, \text{td}_{\text{ks}}) \xleftarrow{\$} \text{TDGen}_{\text{ks}}()$  and returning  $\text{urs}$ . We call this the programmed ROM query. After that query, all further ROM queries are answered with fresh uniformly random bitstrings.

Note that the run of  $\mathcal{A}$  until the programmed ROM query is exactly as the run in the real world, but after the programmed ROM query it can differ.

If this results in a run where  $\mathcal{A}$  makes the  $\text{CHALL}$  query on arguments  $\widetilde{\text{pk}}' = (\text{pk}, \widetilde{\pi}), \text{id}, \widetilde{m}$  with  $\text{Vrfy}_{\text{NIZK}}(\text{urs}, (\text{pp}, \text{id}, \text{pk}), \widetilde{\pi}) = \text{true}$ , we call the run successful and the extractor computes

- $r \xleftarrow{\$} \text{Extr}(\text{urs}, \text{td}_{\text{ks}}, (\text{pp}, \text{id}, \text{pk}), \widetilde{\pi})$ ,
- $(\widehat{\text{pk}}, \widehat{\text{sk}}) \leftarrow \text{VKGen}(\text{pp}, \text{id}; r)$ ,

–  $\sigma \xleftarrow{\$} \text{Sim}(\text{pp}, \text{pk}_S, \text{id}, \widehat{\text{sk}}, m)$ ,

and returns  $\sigma$ . Otherwise, the extractor runs  $\mathcal{A}$  again, up to a maximum of  $\kappa := 2 \ln(2/\gamma)(q_{\mathcal{H}} + 1)/\gamma$  runs. Here,  $\gamma \in (0, 1]$  is a parameter we can choose for a trade-off between runtime and advantage of the reduction. If none of these runs leads to the above case, the extractor outputs  $\perp$ .

We analyze next the probability that the extractor does not receive a valid proof for  $\text{pk}$  in all  $\kappa$  runs. For this analysis we consider an extractor  $\mathcal{E}'$  that uses in the programmed ROM query a uniformly random sampled  $\text{urs}$  (instead of  $\text{urs}$  being sampled via  $(\text{urs}, \text{td}_{\text{ks}}) \xleftarrow{\$} \text{TdGen}_{\text{ks}}()$ ). The knowledge soundness of NIZK guarantees that the probability of a run being successful can change at most with probability  $\varepsilon_{\text{td}}$ .

Next, we describe another, hypothetical extractor  $\mathcal{E}''$  that also runs  $\mathcal{A}$  several times, but simulates these runs less efficiently. Then, we use Lemma 6 to bound the number of runs the hypothetical extractor needs. Finally, we show that the extractor  $\mathcal{E}'$  needs at most as many runs as the hypothetical extractor  $\mathcal{E}''$ .

The hypothetical extractor  $\mathcal{E}''$  works exactly like the extractor  $\mathcal{E}'$ , except that the modified random oracle query is not always the query on  $(s, \text{id}, \text{pk})$ , but always the query on  $(s, \widetilde{\text{id}}, \widetilde{\text{pk}})$  where  $\widetilde{\text{pk}}' = (\widetilde{\text{pk}}, \widetilde{\pi}), \widetilde{\text{id}}, \widetilde{m}$  are the arguments of the CHALL query of  $\mathcal{A}$  in the simulated run. Of course, it might not be known at the time when  $\mathcal{A}$  makes the ROM query on  $(s, \widetilde{\text{id}}, \widetilde{\text{pk}})$  that this is the ROM query used for the CHALL query, but the hypothetical extractor can guess the correct query and run  $\mathcal{A}$  again when that guess was incorrect.

Now, let us use Lemma 6 to bound the number of runs the hypothetical extractor needs. Here, we count as runs only those where the hypothetical extractor programmed the correct random oracle query. Therefore, we set  $\mathcal{D}$  to be the distribution that outputs a transcript of a single run of  $\mathcal{A}$ . That transcript includes the random coins used by  $\mathcal{A}$  and all oracle calls with their respective inputs and outputs and the output of  $\mathcal{A}$ . Let  $X$  be the transcript of the real-world run of the adversary. The function  $h$  maps such a transcript to a truncated transcript that includes all of the randomness and all queries that happen before the ROM query on  $(s, \widetilde{\text{id}}, \widetilde{\text{pk}})$ , where  $\widetilde{\text{pk}}' = (\widetilde{\text{pk}}, \widetilde{\pi}), \widetilde{\text{id}}, \widetilde{m}$  are the arguments of the CHALL query, (excluding this ROM query). Now in each run of  $\mathcal{A}$  by the hypothetical extractor, the resulting transcript  $X'$  is distributed like a fresh transcript conditioned on  $h(X) = h(X')$ . The function  $g$  maps a such a transcript to the index (among all ROM queries) of the ROM query on  $(s, \widetilde{\text{id}}, \widetilde{\text{pk}})$ , where  $\widetilde{\text{pk}}' = (\widetilde{\text{pk}}, \widetilde{\pi}), \widetilde{\text{id}}, \widetilde{m}$  are the arguments of the CHALL query if  $\text{Vrfy}_{\text{NIZK}}(\text{urs}, (\text{pp}, \widetilde{\text{id}}, \widetilde{\text{pk}}), \widetilde{\pi}) = \text{true}$ . Otherwise,  $g$  maps the transcript to a special failure symbol. Note that if a run produces a transcript  $X'$  with  $g(X) = g(X')$ , the extractor successfully obtains a valid proof under a  $\text{urs}$  where it knows the extraction trapdoor. Now Lemma 6 tells us that with probability at least  $1 - \gamma$ , the hypothetical extractor obtains such a successful run among the first  $\kappa = 2 \ln(2/\gamma)(q_{\mathcal{H}} + 1)/\gamma$  runs.

Next, we show that the probability of being successful for a run of the adversary performed by the extractor  $\mathcal{E}'$  is at least as high as for runs performed

by the hypothetical extractor  $\mathcal{E}''$ . Let  $X'$  be a transcript of a run of  $\mathcal{A}$  by  $\mathcal{E}''$  with  $g(X) = g(X')$ . Then, the programmed ROM query has the same index as in the real run  $X$ . Since all queries before the programmed ROM query are identical and the same randomness is used, the programmed ROM query must be on  $(s, \text{id}, \text{pk})$ . Since this is also the ROM query that  $\mathcal{E}'$  programs, the probability that a run by  $\mathcal{E}'$  results in the transcript  $X'$  is at least as high as for a run by  $\mathcal{E}''$ .

If one of the runs of  $\mathcal{A}$  by the real extractor  $\mathcal{E}$  is successful, the extractor will be able to extract with probability at least  $1 - \varepsilon_{\text{ks}}$  a random string  $r$  such that  $(\widehat{\text{pk}}, \widehat{\text{sk}}) \leftarrow \text{VKGen}(\text{pp}, \text{id}; r)$  satisfies  $\widehat{\text{pk}} = \text{pk}$ , because otherwise we can use the extractor to win the knowledge soundness game of NIZK. In particular,  $((\text{id}, \text{pk}), \widehat{\text{sk}}) \in \mathcal{R}_V$ , where  $\mathcal{R}_V$  is the relation for the all verifier key source hiding game.

The all verifier key source hiding notion guarantees now that the adversary  $\mathcal{A}$  and the distinguisher  $\mathcal{D}$  cannot distinguish between a signature generated via  $\text{Sign}(\text{pp}, \text{sk}_S, \text{pk}, \text{id}, m)$  and a signature generated via  $\text{Sim}(\text{pp}, \text{pk}_S, \text{id}, \widehat{\text{sk}}, m)$  (as produced by  $\mathcal{E}$ ).  $\square$

## 5 Impossibility results

In this section we show that it is impossible to achieve simplified malicious source hiding in the standard model and that the EKDH assumption is broken. We apply a similar technique to disprove deniability of X3DH in [Appendix B](#). For these results we give an efficiently sampleable distribution  $\text{AuxPrep}$  of auxiliary inputs such that security does not hold for a randomly sampled auxiliary input. This trivially rules out security in our stronger model with all-quantified auxiliary inputs by taking the auxiliary input from the image of  $\text{AuxPrep}$  that maximizes the advantage of the adversary.

### 5.1 Impossibility of simplified malicious source hiding DVS

We show that a DVS cannot be simultaneously unforgeable and simplified malicious source hiding in the common auxiliary-input model, where the adversary and the extractor get access to the same common auxiliary input. The result assumes the existence of an indistinguishability obfuscator and a puncturable PRF and is on a high level similar to [\[9\]](#).

However, we cannot use the result of [\[9\]](#) in a black-box way, because it is unclear if simplified malicious source hiding DVS imply extractable one-way function. This is in contrast to other “extractable primitives” like SNARKs and, e.g., the knowledge of exponent assumption, which all imply extractable one-way functions [\[9\]](#).

The result does not extend to privately verifiable DVS, because it crucially relies on being able to verify a signature without a secret key.

**Theorem 10.** *Assume there exist a  $(t_{\text{io}}, \varepsilon_{\text{io}})$ -secure and  $\delta$ -correct iO scheme iO and a  $(t_{\text{fp}}, \varepsilon_{\text{fp}})$ -functionality preserving and  $(t_{\text{pr}}, \varepsilon_{\text{pr}})$ -pseudorandom puncturable*



|                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $C_K(\text{pp}, \text{id} \in \mathcal{IDS}):$ <pre> 1 (pk, sk) ← VKGen(pp, id; Eval(K, id))   //where K is a PPRF key 2 return pk AuxPrep(): 3 K ←<sup>\$</sup> KeyGen() 4 return iO(C_K)</pre> | $\mathcal{A}^{\text{CHALL, RANDOM, VKES}}(\text{pp}, \text{pk}_S^*, \text{aux} = C')::$ <pre> 5 m ←<sup>\$</sup> M 6 pk ← C'(\text{pp}, \text{id}^*) 7 σ ←<sup>\$</sup> CHALL(pk, \text{id}^*, m) 8 return st := (\text{pp}, \text{pk}_S^*, \text{pk}, m, σ) D(st = (\text{pp}, \text{pk}_S^*, \text{pk}, m, σ), \text{sk}_S^*): 9 if Vrfy(\text{pp}, \text{pk}_S^*, \text{pk}, m, σ) 10 return 0 11 else 12 return 1</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 9.** On the left-hand side, the circuit  $C_K$  and  $\text{AuxPrep}$ , which obfuscates this circuit. On the right-hand side, the adversary  $(\mathcal{A}, \mathcal{D})$  that breaks simplified malicious source hiding using auxiliary input. The identity  $\text{id}^* \in \mathcal{IDS}$  is a fixed identity.

|                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $C_{\text{id}^*, \text{pk}^*, K_{\text{id}^*}}(\text{pp}, \text{id} \in \mathcal{IDS}):$ <pre> 1 if id = \text{id}^* 2 return \text{pk}^* 3 else 4 (pk, sk) ←<sup>\$</sup> VKGen(pp, id; Eval(K_{\text{id}^*}, id)) 5 return pk</pre> | $\mathcal{B}^{\text{SIGN, VKES}}(\text{pp}, \text{pk}_S):$ <pre> 1 aux ←<sup>\$</sup> AuxPrep() 2 \text{pk}^* ←<sup>\$</sup> VKES(\text{id}^*) 3 K ←<sup>\$</sup> KeyGen() //PPRF 4 K_{\text{id}^*} ←<sup>\$</sup> Puncture(K, \text{id}^*) 5 C' ←<sup>\$</sup> iO(C_{\text{id}^*, \text{pk}^*, K_{\text{id}^*}}) 6 m^* ←<sup>\$</sup> M 7 σ^* ←<sup>\$</sup> E(pp, \text{pk}_S, aux, \emptyset, \text{id}^*, m^*, R) 8 return (m^*, σ^*, \text{id}^*)</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 10.** On the left-hand side, the circuit  $C_{\text{id}^*, \text{pk}^*, K_{\text{id}^*}}$ , which gets obfuscated and then used as auxiliary input from hybrid  $\mathbf{G}_1$  onwards. On the right-hand side, the adversary  $\mathcal{B}$  that breaks unforgeability. The identity  $\text{id}^* \in \mathcal{IDS}$  is a fixed identity and  $R$  is the randomness used to sample  $m^*$ .

*PRF*, then there exists no DVS (in the standard model) that is simultaneously  $(t_{\text{unf}}, \varepsilon_{\text{unf}}, 0, 0)$ -unforgeable and  $(t_{\text{msh}}, t_{\mathcal{E}}, t_{\mathcal{D}}, \varepsilon_{\text{msh}}, 0)$ -simplified malicious source hiding with  $\varepsilon_{\text{unf}} + \varepsilon_{\text{msh}} + \varepsilon_{\text{io}} + \varepsilon_{\text{pr}} + \delta + \varepsilon_{\text{fp}} < 1$  when  $t_{\text{msh}}$  and  $t_{\mathcal{D}}$  are large enough for the simple adversary in [Figure 9](#) and  $t_{\text{unf}}, t_{\text{io}}, t_{\text{fp}}, t_{\text{pr}} \approx t_{\mathcal{E}}$ .

*Proof.* Assume a DVS that is simplified malicious source hiding in the common auxiliary input model. We show how to break unforgeability for such a DVS. Let  $\text{id}^* \in \mathcal{IDS}$  be a fixed identity throughout this proof.

Consider the adversary  $\mathcal{A}$  given in [Figure 9](#) and let  $\mathcal{E}$  be an extractor for this adversary such that  $\text{Adv}_{\text{DVS}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$  is negligible. We now use the extractor to construct an adversary  $\mathcal{B}$  that wins the unforgeability game with overwhelming probability. The adversary  $\mathcal{B}$  is given in [Figure 10](#). Since the adversary  $\mathcal{B}$  uses the extractor on differently distributed inputs compared to how the extractor is called in the simplified malicious source hiding game, we define a sequence of games to argue that the extractor on inputs generated by  $\mathcal{B}$  is almost as successful as in the simplified malicious source hiding game.



To argue that the adversary  $\mathcal{B}$  wins the unforgeability game with overwhelming probability, we need to argue that  $\mathcal{E}$  produces a valid signature with overwhelming probability. For this, we introduce two hybrid games.

Let  $\text{AuxPrep}$  be the auxiliary input sampler described in Figure 9. The first hybrid game  $G_0$  invokes the extractor exactly as it is invoked in the simplified malicious source hiding game with adversary  $\mathcal{A}$  and a (random) auxiliary input  $\text{aux} \leftarrow^{\$} \text{AuxPrep}()$ . The game outputs 1 iff the signature of the extractor verifies.

**Lemma 7 ( $G_0$ ).**  $\Pr[G_0 \Rightarrow 1] = 1 - \mathbb{E}_{\text{aux} \leftarrow^{\$} \text{AuxPrep}()} \text{Adv}_{\text{DVS}, \text{aux}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D}) - \delta \geq 1 - \varepsilon_{\text{msh}} - \delta.$

*Proof.* Assume that evaluation of the obfuscated circuit gives the correct result. This happens with probability  $1 - \delta$ .

Furthermore, when the game  $\mathcal{G}_{\text{DVS}, \text{aux}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$  is played with  $b = 0$  (where the adversary gets an honest signature), the distinguisher always outputs 0 and thus wins the game due to the correctness of DVS. However, if the game is played with  $b = 1$ , the adversary can win at most with probability  $\text{Adv}_{\text{DVS}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$ , since the advantage of  $\mathcal{A}$  is  $\text{Adv}_{\text{DVS}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$ . Hence, in the case  $b = 1$  the adversary has to output 0 with probability at least  $1 - \text{Adv}_{\text{DVS}}^{\text{s-mal-sh}}(\mathcal{A}, \mathcal{D})$  and in this case the signature of the extractor did verify.  $\square$

In the next hybrid game  $G_1$ , the auxiliary input is changed to an obfuscation of the circuit  $C_{\text{id}^*, \text{pk}^*, K_{\text{id}^*}}$  given in Figure 10 with the following constants:

- $K_{\text{id}^*} \leftarrow^{\$} \text{Puncture}(K, \text{id}^*)$  where  $K \leftarrow^{\$} \text{KeyGen}()$
- $\text{pk}^*$  where  $(\text{pk}^*, \text{sk}^*) \leftarrow \text{VKGen}(\text{pp}, \text{id}^*; \text{Eval}(K, \text{id}^*))$

**Lemma 8 ( $G_0 \rightsquigarrow G_1$ ).**  $|\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \varepsilon_{\text{io}} + \varepsilon_{\text{fp}}.$

*Proof.* The obfuscated circuits in  $G_0$  and  $G_1$  are functionally equivalent, because:

- The keys for all users except  $\text{id}^*$  are generated in the same way, except that the punctured key  $K_{\text{id}^*}$  is used instead of  $K$ . Since the PPRF is not evaluated on  $\text{id}^*$  here, this is functionally equivalent (except with probability  $\varepsilon_{\text{fp}}$ ) because the PPRF is  $(t_{\text{fp}}, \varepsilon_{\text{fp}})$ -functionality preserving.
- The key for user  $\text{id}^*$  is computed in the same way as before, but the result is hard-coded in the obfuscated program.

With this, the reduction to iO is straightforward.  $\square$

In the next hybrid  $G_2$ , we compute  $\text{pk}^*$  as  $(\text{pk}^*, \text{sk}^*) \leftarrow^{\$} \text{VKGen}(\text{pp}, \text{id}^*)$  (with true randomness) instead of as  $(\text{pk}^*, \text{sk}^*) \leftarrow \text{VKGen}(\text{pp}, \text{id}^*; \text{Eval}(K, \text{id}^*))$ .

**Lemma 9 ( $G_1 \rightsquigarrow G_2$ ).** *There exists a PPT adversary  $\mathcal{D}$  with*

$$|\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \text{Adv}_{\text{PPRF}}^{\text{pprf}}(\mathcal{D}) \leq \varepsilon_{\text{pr}}.$$

*Proof.* This is a straightforward reduction to the security of PPRF. This security game gives us the punctured key  $K_{\text{id}^*}$ , which gets hard-coded in the obfuscated program. The public key  $\text{pk}^*$  is generated with the challenge value obtained from the PPRF security game, which is either  $\text{Eval}(K, \text{id}^*)$  (corresponding to game  $G_1$ ) or uniformly random (corresponding to game  $G_2$ )  $\square$

**Lemma 10 ( $G_2$ ).**  $\Pr[G_2 \Rightarrow 1] = \text{Adv}_{\text{DVS}}^{\text{uf}}(\mathcal{B}) \leq \varepsilon_{\text{unf}}$

*Proof.* The adversary  $\mathcal{B}$  and the game  $G_2$  invoke the extractor  $\mathcal{E}$  on identically distributed values. In particular, in both invocations  $\text{pp}$ ,  $\text{pk}_S$  and  $\text{pk}^*$  (hard-coded in the obfuscated program) are honestly generated. The game  $G_2$  outputs 1 if the signature returned by the obfuscated program verifies, which is exactly when  $\mathcal{B}$  wins the unforgeability game.  $\square$

Combining Lemmata 7 – 10 yields  $\varepsilon_{\text{unf}} \geq 1 - \varepsilon_{\text{msh}} - \varepsilon_{\text{io}} - \varepsilon_{\text{pr}} - \delta - \varepsilon_{\text{fp}}$ .  $\square$

*Remark 5.* In our impossibility result, the auxiliary input depends on the specific scheme (concretely, the VKGen algorithm). This dependency on the specific scheme can be easily avoided by obfuscating a universal circuit of sufficient size instead.

*Remark 6.* This result shows that a publicly verifiable DVS cannot be unforgeable and simplified malicious source hiding in the standard model. Since ring signatures imply publicly verifiable DVS, our impossibility result also rules out unforgeable and simplified malicious source hiding ring signatures in the standard model.

## 5.2 Disproving the (E)KDH assumption

In this section we show that the (E)KDH assumption with arbitrary auxiliary input does not hold under very mild conditions on the group, that are satisfied by all groups interesting for cryptography (e.g. elliptic curves). The result also implies that the KDH assumption (which is stronger, since the adversary gets more information) is not true.

In the following, let  $\mathcal{G} = (\mathbb{G}, g, q)$  be a description of a group  $\mathbb{G}$  of prime-order  $q$  with generator  $g$  and function  $\text{DH} : \mathcal{G} \times \mathcal{G} \rightarrow \mathcal{G}$  with  $(g^a, g^b) \mapsto g^{ab}$ .

**Definition 20 (Hinted CDH problem).** Let  $\mathcal{G}$  be a cyclic group with generator  $g$ . We say that the  $(t, \varepsilon, \eta)$ -hinted computational Diffie-Hellman (CDH) assumption holds over group  $\mathcal{G}$  if for every adversary  $\mathcal{A}$  running in time at most  $t$  it holds that  $\Pr_{a,b \xleftarrow{\$} \mathbb{Z}_q} [\mathcal{A}(\mathcal{G}, g^a, g^b, \lfloor b/\eta \rfloor) = \text{DH}(g^a, g^b)] \leq \varepsilon$ .

The hinted CDH problem with  $\eta = 1$  is trivially efficiently solvable in any group where the group operation is efficiently computable. On the other hand, for  $\eta = q$  the problem is equivalent to the regular CDH problem, which is hard in groups used in cryptography. Informally speaking, our attack requires that the hardness of the hinted CDH problem increases continuously with increasing  $\eta$ , which is a very mild assumption as we explain below.

**Definition 21 (KDH and EKDH Assumptions [71]).** Let  $\mathcal{G}$  be a cyclic group with generator  $g$ . Let  $\mathcal{A}$  be any algorithm running in time  $t_{\mathcal{A}}$  which runs on input  $\mathcal{G}$ ,  $U \in \mathcal{G}$ , and  $\text{aux} \in \{0, 1\}^*$  and outputs  $Z \in \mathcal{G}$ . We denote with  $Z \leftarrow \mathcal{A}(\mathcal{G}, U, \text{aux}; r)$  the output of running  $\mathcal{A}$  on input  $\mathcal{G}, U, \text{aux}$  with random coins  $r$ .

We say that the  $(t_{\mathcal{A}}, t_{\mathcal{E}}, t_{\mathcal{D}}, \varepsilon_{\mathcal{D}})$ -Knowledge of DH (KDH) assumption holds over group  $\mathcal{G}$  iff for every  $\mathcal{A}$ , there exists an extractor  $\mathcal{E}$  such that for all auxiliary inputs  $\text{aux} \in \{0, 1\}^*$  and uniformly random  $U \xleftarrow{\$} \mathcal{G}$

- $\mathcal{E}$  runs on input  $\mathcal{G}, U, \mathbf{aux}$ , and  $r$  in time  $t_{\mathcal{E}}$  and outputs  $\hat{Z} \in \mathcal{G}$  or  $\perp$
- If  $\mathcal{E}(\mathcal{G}, U, \mathbf{aux}, r) \neq \perp$  then  $\hat{Z} = \text{DH}(U, Z)$
- For every algorithm  $C$  running in time  $t_{\mathcal{D}}$ , we have that  $\Pr[C(\mathcal{G}, U, r, \mathbf{aux}) = \text{DH}(U, Z) \mid \mathcal{E}(\mathcal{G}, U, \mathbf{aux}, r) = \perp] \leq \varepsilon_{\mathcal{D}}$ , where  $Z \leftarrow \mathcal{A}(\mathcal{G}, U, \mathbf{aux}; r)$  and the probability is taken over the coins of  $C$  and the randomness used to sample  $r$ .

We say that the  $(t_{\mathcal{A}}, t_{\mathcal{E}}, t_{\mathcal{D}}, \varepsilon_{\mathcal{D}})$ -Extended Knowledge of DH (EKDH) assumption holds over group  $\mathcal{G}$  if the same holds for an algorithm  $\mathcal{A}$  that does not receive  $U$  as input.

The original definition of [71] is ambiguous in two aspects, but our attack breaks this assumption in all meaningful interpretations. First, they define security with respect to a class of auxiliary inputs  $\mathbf{AUX}$ , but never specify for which classes  $\mathbf{AUX}$  they conjecture this assumption to hold. For simplicity we have stated the definition for  $\mathbf{AUX} = \{0, 1\}^*$ , but in our attack the auxiliary input is extremely simple. It can be described by less than  $2\lceil \log q \rceil$  bits, where  $q$  is the order of  $\mathcal{G}$ .

Secondly, it is not clear in their definition what  $\mathbf{aux}$  is and what is allowed to depend on  $\mathbf{aux}$ , apart from the condition  $\mathbf{aux} \in \mathbf{AUX}$ . We interpret this as security has to hold for all  $\mathbf{aux} \in \mathbf{AUX}$  and that the extractor (and thus also the adversary) is not allowed to depend on  $\mathbf{aux}$ . If instead the extractor is allowed to depend on  $\mathbf{aux}$ , then the auxiliary input would be meaningless because it can be replaced by a string hard-coded in the description of  $\mathcal{A}$ . The version where  $\mathbf{aux}$  is sampled uniformly at random from  $\mathbf{AUX}$  and  $\mathbf{AUX}$  is an arbitrary set is equivalent to the version where  $\mathbf{aux}$  is all-quantified, since  $\mathbf{AUX}$  can be the singleton set that just contains the value  $\mathbf{aux}$  that maximizes the success probability for the adversary. Finally, the version where  $\mathbf{aux}$  is sampled uniformly at random from a fixed set  $\mathbf{AUX}$  is a weaker assumption, but our attack can still break it for a very simple set  $\mathbf{AUX}$  (in particular from a set we can very efficiently sample from).

The work [71] justifies the KDH assumption by reducing it to the knowledge of exponent assumption (KEA) and another novel assumption called knowledge of discrete logarithm in the common auxiliary input model. That result is meaningless, because the KEA assumption does not hold in the common auxiliary input model [9]. Curiously, the authors of [71] explicitly mention [9] in their work and claim to bypass their impossibility result with the (E)KDH assumption.

Our proof uses the following general lower bound for a conditional probability.

**Fact 11.** Let  $A$  and  $C$  be events with  $\Pr[C] > 0$ . Then  $\Pr[A \mid C] \geq 1 - \frac{1 - \Pr[A]}{\Pr[C]}$ .

$$\textit{Proof.} \quad \Pr[A \mid C] = \frac{\Pr[A \wedge C]}{\Pr[C]} = \frac{\Pr[A] - \Pr[A \wedge \neg C]}{\Pr[C]} \geq \frac{\Pr[A] - \Pr[\neg C]}{\Pr[C]} = \frac{\Pr[A] - 1 + \Pr[C]}{\Pr[C]} = 1 - \frac{1 - \Pr[A]}{\Pr[C]} \quad \square$$

In the following attack we use the auxiliary input to give a group element and a hint about the corresponding exponent to the adversary, who outputs this group element as its public key. Now, both the extractor and the distinguisher (the algorithm  $C$  in case the extractor fails) need to solve the same hinted CDH challenge, but with different runtime and advantage.

**Theorem 12.** *If  $\mathcal{G} = (\mathbb{G}, g, q)$  is a group where the  $(t, \varepsilon, \eta)$ -hinted CDH assumption holds for  $\varepsilon < 1$ , but the  $(t', \varepsilon', \eta)$ -hinted CDH assumption (for some  $t < t'$ ) does not hold, then the  $(t_A, t, t', 1 - (1 - \varepsilon')/(1 - \varepsilon))$ -EKDH assumption does not hold in  $\mathcal{G}$ .*

*Proof.* Consider the variant where the auxiliary input is generated by sampling  $b \xleftarrow{\$} \mathbb{Z}_q$  and returning  $\mathbf{aux} = (g^b, \lfloor b/\eta \rfloor)$ ; and the adversary  $\mathcal{A}$  playing against the EKDH assumption on input  $(\mathcal{G}, \mathbf{aux} = (Z = g^b, b' = \lfloor b/\eta \rfloor))$  outputs  $Z$ .

The extractor  $\mathcal{E}$  for this adversary then has to solve the hinted CDH problem on  $U$  and  $Z$  with parameter  $\eta$  and has time  $t$ . Since we assume the  $(t, \varepsilon, \eta)$ -hinted CDH assumption, the extractor can output  $\text{DH}(U, Z)$  with probability at most  $\varepsilon$ .

On the other hand, since we assume  $(t', \varepsilon', \eta)$ -hinted CDH to not hold, there exists an algorithm  $\mathcal{A}_{\text{HCDH}}$  with  $\Pr[\mathcal{A}_{\text{HCDH}}(\mathcal{G}, U, Z, b') = \text{DH}(U, Z)] > \varepsilon'$ . Let  $C(\mathcal{G}, U, r, \mathbf{aux} = (Z, b'))$  be the circuit that ignores  $r$  and runs  $\mathcal{A}_{\text{HCDH}}(\mathcal{G}, U, Z, b')$ .

Then, we get by applying [Fact 11](#)

$$\begin{aligned} \Pr[C(\mathcal{G}, U, r, \mathbf{aux} = (Z, b')) = \text{DH}(U, Z) \mid \mathcal{E}(\mathcal{G}, U, \mathbf{aux}, r) = \perp] \\ \geq 1 - \frac{1 - \Pr[C(\mathcal{G}, U, r, \mathbf{aux}) = \text{DH}(U, Z)]}{\Pr[\mathcal{E}(\mathcal{G}, U, \mathbf{aux}, r) = \perp]} \geq 1 - \frac{1 - \varepsilon'}{1 - \varepsilon}, \end{aligned}$$

where the probability is taken over sampling  $U, r, \mathbf{aux}$  and the random coins of  $C$  and  $\mathcal{E}$ .

This implies that there exists an auxiliary input  $\mathbf{aux}$  such that

$$\Pr[C(\mathcal{G}, U, r, \mathbf{aux} = (Z, b')) = \text{DH}(U, Z) \mid \mathcal{E}(\mathcal{G}, U, \mathbf{aux}, r) = \perp] \geq 1 - \frac{1 - \varepsilon'}{1 - \varepsilon},$$

where the probability is taken over sampling  $U, r$  and the random coins of  $C$  and  $\mathcal{E}$ .  $\square$

Note that the result of [Theorem 12](#) is meaningful if  $\varepsilon < \varepsilon'$ . We next argue that in groups typically used in cryptography we find for arbitrary  $t$  and  $t'$  with  $t < t'$  a suitable hint size  $\eta$  such that the  $(t, \varepsilon, \eta)$ -hinted CDH assumption holds, but the  $(t', \varepsilon', \eta)$ -hinted CDH assumption does not hold.

In generic groups (or any group where the best known algorithm to solve the hinted CDH assumption is generic, such as elliptic curves) it is easy to see that the best algorithm to solve the hinted CDH assumption is to compute the discrete log of the second group element by searching through the space of possible exponents with the baby-step-giant-step algorithm [\[61, 63\]](#), which takes  $c\sqrt{q/h}$  steps for some constant  $c$ . By setting  $t' = c\sqrt{h}$ , we ensure that the  $(t', \varepsilon', \eta)$ -hinted CDH assumption does not hold for any  $\varepsilon' < 1$ , but the  $(t, \varepsilon, \eta)$ -hinted CDH assumption holds for any  $t < t'$  for a suitable  $\varepsilon < 1$  which leads to an attack on the EKDH assumption with advantage  $\varepsilon_{\mathcal{D}} = 1$ .

This analysis also works for groups where faster algorithms than generic ones are available, as long as we can find a hint size  $\eta$  such that time  $t$  is just enough to solve the hinted CDH problem with probability 1. In particular, the argument fails if the time  $t$  is enough to break CDH without any hint, but this setting is not interesting for cryptography anyway.

*Acknowledgments.* We thank Nils Fleischhacker and Guilherme Rito for discussions on an early version of this work. They also study deniable authentication in a model where the receiver is not trusted to generate their public key honestly in a concurrent work to this one [33].

R.F. was supported by the German Federal Ministry of Education and Research and the Hessian Ministry of Higher Education, Research, Science and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE.

## References

- [1] Joël Alwen, Bruno Blanchet, Eduard Hauck, Eike Kiltz, Benjamin Lipp, and Doreen Riepel. “Analysing the HPKE Standard”. In: *EUROCRYPT 2021, Part I*. Ed. by Anne Canteaut and François-Xavier Standaert. Vol. 12696. LNCS. Springer, Cham, Oct. 2021, pp. 87–116. DOI: [10.1007/978-3-030-77870-5\\_4](https://doi.org/10.1007/978-3-030-77870-5_4).
- [2] Arasu Arun, Joseph Bonneau, and Jeremy Clark. “Short-lived Zero-Knowledge Proofs and Signatures”. In: *ASIACRYPT 2022, Part III*. Ed. by Shweta Agrawal and Dongdai Lin. Vol. 13793. LNCS. Springer, Cham, Dec. 2022, pp. 487–516. DOI: [10.1007/978-3-031-22969-5\\_17](https://doi.org/10.1007/978-3-031-22969-5_17).
- [3] Maryam Rajabzadeh Asaar, Mahmoud Salmasizadeh, and Mohammad Reza Aref. *Code-based Strong Designated Verifier Signatures: Security Analysis and a New Construction*. Cryptology ePrint Archive, Report 2016/779. 2016. URL: <https://eprint.iacr.org/2016/779>.
- [4] Hafsa Assidi and El Mamoun Souidi. “Strong Designated Verifier Signature Based on the Rank Metric”. In: *Information Security Theory and Practice - 13th IFIP WG 11.2 International Conference, WISTP 2019, Paris, France, December 11-12, 2019, Proceedings*. Ed. by Maryline Laurent and Thanassis Giannetsos. Vol. 12024. Lecture Notes in Computer Science. Springer, 2019, pp. 85–102. DOI: [10.1007/978-3-030-41702-4\\_6](https://doi.org/10.1007/978-3-030-41702-4_6).
- [5] Thomas Aulbach, Samed Düzl , Michael Meyer, Patrick Struck, and Maximiliane Weish upl. “Hash Your Keys Before Signing - BUFF Security of the Additional NIST PQC Signatures”. In: *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*. Ed. by Markku-Juhani Saari nen and Daniel Smith-Tone. Springer, Cham, June 2024, pp. 301–335. DOI: [10.1007/978-3-031-62746-0\\_13](https://doi.org/10.1007/978-3-031-62746-0_13).
- [6] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. “On the (Im)possibility of Obfuscating Programs”. In: *CRYPTO 2001*. Ed. by Joe Kilian. Vol. 2139. LNCS. Springer, Berlin, Heidelberg, Aug. 2001, pp. 1–18. DOI: [10.1007/3-540-44647-8\\_1](https://doi.org/10.1007/3-540-44647-8_1).
- [7] Gabrielle Beck, Arka Rai Choudhuri, Matthew Green, Abhishek Jain, and Pratyush Ranjan Tiwari. “Time-Deniable Signatures”. In: *PoPETs 2023.3* (July 2023), pp. 79–102. DOI: [10.56553/popets-2023-0071](https://doi.org/10.56553/popets-2023-0071).
- [8] Mihir Bellare and Phillip Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *ACM CCS 93*. Ed. by Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby. ACM Press, Nov. 1993, pp. 62–73. DOI: [10.1145/168588.168596](https://doi.org/10.1145/168588.168596).
- [9] Nir Bitansky, Ran Canetti, Omer Paneth, and Alon Rosen. “On the existence of extractable one-way functions”. In: *46th ACM STOC*. Ed. by David B. Shmoys. ACM Press, 2014, pp. 505–514. DOI: [10.1145/2591796.2591859](https://doi.org/10.1145/2591796.2591859).

- [10] Dan Boneh, Amit Sahai, and Brent Waters. “Functional Encryption: Definitions and Challenges”. In: *TCC 2011*. Ed. by Yuval Ishai. Vol. 6597. LNCS. Springer, Berlin, Heidelberg, Mar. 2011, pp. 253–273. DOI: [10.1007/978-3-642-19571-6\\_16](https://doi.org/10.1007/978-3-642-19571-6_16).
- [11] Nikita Borisov, Ian Goldberg, and Eric A. Brewer. “Off-the-record communication, or, why not to use PGP”. In: *Proceedings of the 2004 ACM Workshop on Privacy in the Electronic Society, WPES 2004, Washington, DC, USA, October 28, 2004*. Ed. by Vijay Atluri, Paul F. Syverson, and Sabrina De Capitani di Vimercati. ACM, 2004, pp. 77–84. DOI: [10.1145/1029179.1029200](https://doi.org/10.1145/1029179.1029200).
- [12] Pedro Branco, Nico Döttling, and Jesko Dujmovic. “Rate-1 Incompressible Encryption from Standard Assumptions”. In: *TCC 2022, Part II*. Ed. by Eike Kiltz and Vinod Vaikuntanathan. Vol. 13748. LNCS. Springer, Cham, Nov. 2022, pp. 33–69. DOI: [10.1007/978-3-031-22365-5\\_2](https://doi.org/10.1007/978-3-031-22365-5_2).
- [13] Jacqueline Brendel, Rune Fiedler, Felix Günther, Christian Janson, and Douglas Stebila. “Post-quantum Asynchronous Deniable Key Exchange and the Signal Handshake”. In: *PKC 2022, Part II*. Ed. by Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe. Vol. 13178. LNCS. Springer, Cham, Mar. 2022, pp. 3–34. DOI: [10.1007/978-3-030-97131-1\\_1](https://doi.org/10.1007/978-3-030-97131-1_1).
- [14] Jacqueline Brendel, Marc Fischlin, Felix Günther, Christian Janson, and Douglas Stebila. “Towards Post-Quantum Security for Signal’s X3DH Handshake”. In: *SAC 2020*. Ed. by Orr Dunkelman, Michael J. Jacobson Jr., and Colin O’Flynn. Vol. 12804. LNCS. Springer, Cham, Oct. 2020, pp. 404–430. DOI: [10.1007/978-3-030-81652-0\\_16](https://doi.org/10.1007/978-3-030-81652-0_16).
- [15] Jie Cai, Han Jiang, Pingyuan Zhang, Zhihua Zheng, Hao Wang, Guangshi Lü, and Qiuliang Xu. “ID-Based Strong Designated Verifier Signature over  $\mathcal{R}$ -SIS Assumption”. In: *Security and Communication Networks 2019 (2019)*, 9678095:1–9678095:8. DOI: [10.1155/2019/9678095](https://doi.org/10.1155/2019/9678095).
- [16] Ran Canetti, Oded Goldreich, and Shai Halevi. “The Random Oracle Methodology, Revisited (Preliminary Version)”. In: *30th ACM STOC*. ACM Press, May 1998, pp. 209–218. DOI: [10.1145/276698.276741](https://doi.org/10.1145/276698.276741).
- [17] Suvradip Chakraborty, Dennis Hofheinz, Ueli Maurer, and Guilherme Rito. “Deniable Authentication When Signing Keys Leak”. In: *EUROCRYPT 2023, Part III*. Ed. by Carmit Hazay and Martijn Stam. Vol. 14006. LNCS. Springer, Cham, Apr. 2023, pp. 69–100. DOI: [10.1007/978-3-031-30620-4\\_3](https://doi.org/10.1007/978-3-031-30620-4_3).
- [18] Daniel Collins, Loïs Huguenin-Dumittan, Ngoc Khanh Nguyen, Nicolas Rolin, and Serge Vaudenay. “K-Waay: Fast and Deniable Post-Quantum X3DH without Ring Signatures”. In: *USENIX Security 2024*. Ed. by Davide Balzarotti and Wenyan Xu. USENIX Association, Aug. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/collins>.
- [19] Cas Cremers, Alexander Dax, and Niklas Medinger. “Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols”. In: *ACM CCS 2024*. Ed. by Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie. ACM Press, Oct. 2024, pp. 1046–1060. DOI: [10.1145/3658644.3670283](https://doi.org/10.1145/3658644.3670283).
- [20] Cas Cremers, Samed Düzlül, Rune Fiedler, Marc Fischlin, and Christian Janson. “BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures”. In: *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 1696–1714. DOI: [10.1109/SP40001.2021.00093](https://doi.org/10.1109/SP40001.2021.00093).
- [21] Ivan Damgård. “Towards Practical Public Key Systems Secure Against Chosen Ciphertext Attacks”. In: *CRYPTO’91*. Ed. by Joan Feigenbaum. Vol. 576.



- LNCS. Springer, Berlin, Heidelberg, Aug. 1992, pp. 445–456. DOI: [10.1007/3-540-46766-1\\_36](https://doi.org/10.1007/3-540-46766-1_36).
- [22] Mario Di Raimondo and Rosario Gennaro. “New Approaches for Deniable Authentication”. In: *ACM CCS 2005*. Ed. by Vijayalakshmi Atluri, Catherine Meadows, and Ari Juels. ACM Press, Nov. 2005, pp. 112–121. DOI: [10.1145/1102120.1102137](https://doi.org/10.1145/1102120.1102137).
  - [23] Mario Di Raimondo, Rosario Gennaro, and Hugo Krawczyk. “Deniable authentication and key exchange”. In: *ACM CCS 2006*. Ed. by Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati. ACM Press, 2006, pp. 400–409. DOI: [10.1145/1180405.1180454](https://doi.org/10.1145/1180405.1180454).
  - [24] Samuel Dobson and Steven D. Galbraith. “Post-Quantum Signal Key Agreement from SIDH”. In: *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022*. Ed. by Jung Hee Cheon and Thomas Johansson. Springer, Cham, Sept. 2022, pp. 422–450. DOI: [10.1007/978-3-031-17234-2\\_20](https://doi.org/10.1007/978-3-031-17234-2_20).
  - [25] Danny Dolev, Cynthia Dwork, and Moni Naor. “Non-Malleable Cryptography (Extended Abstract)”. In: *23rd ACM STOC*. ACM Press, May 1991, pp. 542–552. DOI: [10.1145/103418.103474](https://doi.org/10.1145/103418.103474).
  - [26] Jelle Don, Serge Fehr, Yu-Hsuan Huang, Jyun-Jie Liao, and Patrick Struck. “Hide-and-Seek and the Non-resignability of the BUFF Transform”. In: *TCC 2024, Part III*. Ed. by Elette Boyle and Mohammad Mahmoody. Vol. 15366. LNCS. Springer, Cham, Dec. 2024, pp. 347–370. DOI: [10.1007/978-3-031-78020-2\\_12](https://doi.org/10.1007/978-3-031-78020-2_12).
  - [27] Jelle Don, Serge Fehr, Yu-Hsuan Huang, and Patrick Struck. “On the (In)Security of the BUFF Transform”. In: *CRYPTO 2024, Part I*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14920. LNCS. Springer, Cham, Aug. 2024, pp. 246–275. DOI: [10.1007/978-3-031-68376-3\\_8](https://doi.org/10.1007/978-3-031-68376-3_8).
  - [28] Samed Düzl , Rune Fiedler, and Marc Fischlin. “BUFFing FALCON Without Increasing the Signature Size”. In: *SAC 2024, Part I*. Ed. by Maria Eichlseder and S bastien Gambs. Vol. 15516. LNCS. Springer, Cham, Aug. 2024, pp. 131–150. DOI: [10.1007/978-3-031-82852-2\\_6](https://doi.org/10.1007/978-3-031-82852-2_6).
  - [29] Cynthia Dwork, Moni Naor, and Amit Sahai. “Concurrent Zero-Knowledge”. In: *30th ACM STOC*. ACM Press, May 1998, pp. 409–418. DOI: [10.1145/276698.276853](https://doi.org/10.1145/276698.276853).
  - [30] Uriel Feige and Adi Shamir. “Zero Knowledge Proofs of Knowledge in Two Rounds”. In: *CRYPTO’89*. Ed. by Gilles Brassard. Vol. 435. LNCS. Springer, New York, Aug. 1990, pp. 526–544. DOI: [10.1007/0-387-34805-0\\_46](https://doi.org/10.1007/0-387-34805-0_46).
  - [31] Amos Fiat and Adi Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *CRYPTO’86*. Ed. by Andrew M. Odlyzko. Vol. 263. LNCS. Springer, Berlin, Heidelberg, Aug. 1987, pp. 186–194. DOI: [10.1007/3-540-47721-7\\_12](https://doi.org/10.1007/3-540-47721-7_12).
  - [32] Rune Fiedler and Christian Janson. “A Deniability Analysis of Signal’s Initial Handshake PQXDH”. In: *PoPETs 2024.4* (Oct. 2024), pp. 907–928. DOI: [10.56553/popets-2024-0148](https://doi.org/10.56553/popets-2024-0148).
  - [33] Nils Fleischh cker and Guilherme Rito. *Guaranteeing a Dishonest Party’s Knowledge (Or: Setup Requirements for Deniable Authentication)*. Cryptology ePrint Archive, Paper 2025/1922. 2025. URL: <https://eprint.iacr.org/2025/1922>.
  - [34] Shafi Goldwasser and Yael Tauman Kalai. “On the (In)security of the Fiat-Shamir Paradigm”. In: *44th FOCS*. IEEE Computer Society Press, Oct. 2003, pp. 102–115. DOI: [10.1109/SFCS.2003.1238185](https://doi.org/10.1109/SFCS.2003.1238185).
  - [35] Keitaro Hashimoto, Shuichi Katsumata, Kris Kwiatkowski, and Thomas Prest. “An Efficient and Generic Construction for Signal’s Handshake (X3DH): Post-

- Quantum, State Leakage Secure, and Deniable”. In: *PKC 2021, Part II*. Ed. by Juan Garay. Vol. 12711. LNCS. Springer, Cham, May 2021, pp. 410–440. DOI: [10.1007/978-3-030-75248-4\\_15](https://doi.org/10.1007/978-3-030-75248-4_15).
- [36] Dennis Hofheinz, Tibor Jager, Dakshita Khurana, Amit Sahai, Brent Waters, and Mark Zhandry. “How to Generate and Use Universal Samplers”. In: *ASIACRYPT 2016, Part II*. Ed. by Jung Hee Cheon and Tsuyoshi Takagi. Vol. 10032. LNCS. Springer, Berlin, Heidelberg, Dec. 2016, pp. 715–744. DOI: [10.1007/978-3-662-53890-6\\_24](https://doi.org/10.1007/978-3-662-53890-6_24).
- [37] Dennis Hofheinz, Julia Kastner, and Karen Klein. “The Power of Undirected Rewindings for Adaptive Security”. In: *CRYPTO 2023, Part II*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14082. LNCS. Springer, Cham, Aug. 2023, pp. 725–758. DOI: [10.1007/978-3-031-38545-2\\_24](https://doi.org/10.1007/978-3-031-38545-2_24).
- [38] Qiong Huang, Guomin Yang, Duncan S. Wong, and Willy Susilo. *Efficient Strong Designated Verifier Signature Schemes without Random Oracles or Delegatability*. Cryptology ePrint Archive, Report 2009/518. 2009. URL: <https://eprint.iacr.org/2009/518>.
- [39] Andreas Hülsing and Fiona Johanna Weber. “Epochal Signatures for Deniable Group Chats”. In: *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 1677–1695. DOI: [10.1109/SP40001.2021.00058](https://doi.org/10.1109/SP40001.2021.00058).
- [40] Aayush Jain, Huijia Lin, and Amit Sahai. “Indistinguishability Obfuscation from LPN over  $\mathbb{F}_p$ , DLIN, and PRGs in  $NC^0$ ”. In: *EUROCRYPT 2022, Part I*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13275. LNCS. Springer, Cham, 2022, pp. 670–699. DOI: [10.1007/978-3-031-06944-4\\_23](https://doi.org/10.1007/978-3-031-06944-4_23).
- [41] Aayush Jain, Huijia Lin, and Amit Sahai. “Indistinguishability obfuscation from well-founded assumptions”. In: *53rd ACM STOC*. Ed. by Samir Khuller and Virginia Vassilevska Williams. ACM Press, June 2021, pp. 60–73. DOI: [10.1145/3406325.3451093](https://doi.org/10.1145/3406325.3451093).
- [42] Markus Jakobsson, Kazue Sako, and Russell Impagliazzo. “Designated Verifier Proofs and Their Applications”. In: *EUROCRYPT’96*. Ed. by Ueli M. Maurer. Vol. 1070. LNCS. Springer, Berlin, Heidelberg, May 1996, pp. 143–154. DOI: [10.1007/3-540-68339-9\\_13](https://doi.org/10.1007/3-540-68339-9_13).
- [43] Julia Kastner, Julian Loss, and Jiayu Xu. “The Abe-Okamoto Partially Blind Signature Scheme Revisited”. In: *ASIACRYPT 2022, Part IV*. Ed. by Shweta Agrawal and Dongdai Lin. Vol. 13794. LNCS. Springer, Cham, Dec. 2022, pp. 279–309. DOI: [10.1007/978-3-031-22972-5\\_10](https://doi.org/10.1007/978-3-031-22972-5_10).
- [44] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. *How to Prove False Statements: Practical Attacks on Fiat-Shamir*. Cryptology ePrint Archive, Report 2025/118. 2025. URL: <https://eprint.iacr.org/2025/118>.
- [45] Ehren Kret and Rolfe Schmidt. *The PQXDH key agreement protocol*. <https://signal.org/docs/specifications/pqxdh/>. September 2023.
- [46] Caroline Kudla and Kenneth G. Paterson. “Non-interactive Designated Verifier Proofs and Undeniable Signatures”. In: *10th IMA International Conference on Cryptography and Coding*. Ed. by Nigel P. Smart. Vol. 3796. LNCS. Springer, Berlin, Heidelberg, Dec. 2005, pp. 136–154. DOI: [10.1007/11586821\\_10](https://doi.org/10.1007/11586821_10).
- [47] Fabien Laguillaumie and Damien Vergnaud. “Designated Verifier Signatures: Anonymity and Efficient Construction from Any Bilinear Map”. In: *SCN 04*. Ed. by Carlo Blundo and Stelvio Cimato. Vol. 3352. LNCS. Springer, Berlin, Heidelberg, Sept. 2005, pp. 105–119. DOI: [10.1007/978-3-540-30598-9\\_8](https://doi.org/10.1007/978-3-540-30598-9_8).



- [48] Yong Li, Helger Lipmaa, and Dingyi Pei. “On Delegatability of Four Designated Verifier Signatures”. In: *ICICS 05*. Ed. by Sihan Qing, Wenbo Mao, Javier López, and Guilin Wang. Vol. 3783. LNCS. Springer, Berlin, Heidelberg, Dec. 2005, pp. 61–71. DOI: [10.1007/11602897\\_6](https://doi.org/10.1007/11602897_6).
- [49] Yong Li, Willy Susilo, Yi Mu, and Dingyi Pei. “Designated Verifier Signature: Definition, Framework and New Constructions”. In: *Ubiquitous Intelligence and Computing, 4th International Conference, UIC 2007, Hong Kong, China, July 11-13, 2007, Proceedings*. Ed. by Jadwiga Indulska, Jianhua Ma, Laurence Tianruo Yang, Theo Ungerer, and Jiannong Cao. Vol. 4611. Lecture Notes in Computer Science. Springer, 2007, pp. 1191–1200. DOI: [10.1007/978-3-540-73549-6\\_116](https://doi.org/10.1007/978-3-540-73549-6_116).
- [50] Helger Lipmaa, Guilin Wang, and Feng Bao. “Designated Verifier Signature Schemes: Attacks, New Security Notions and a New Construction”. In: *ICALP 2005*. Ed. by Luís Caires, Giuseppe F. Italiano, Luís Monteiro, Catuscia Palamidessi, and Moti Yung. Vol. 3580. LNCS. Springer, Berlin, Heidelberg, July 2005, pp. 459–471. DOI: [10.1007/11523468\\_38](https://doi.org/10.1007/11523468_38).
- [51] Moxie Marlinspike and Trevor Perrin. *The X3DH key agreement protocol*. <https://signal.org/docs/specifications/x3dh/>. November 2016.
- [52] Jesper Buus Nielsen. “Separating Random Oracle Proofs from Complexity Theoretic Proofs: The Non-committing Encryption Case”. In: *CRYPTO 2002*. Ed. by Moti Yung. Vol. 2442. LNCS. Springer, Berlin, Heidelberg, Aug. 2002, pp. 111–126. DOI: [10.1007/3-540-45708-9\\_8](https://doi.org/10.1007/3-540-45708-9_8).
- [53] Geontae Noh and Ik Rae Jeong. “Strong designated verifier signature scheme from lattices in the standard model”. In: *Security and Communication Networks* 9.18 (2016), pp. 6202–6214. DOI: [10.1002/SEC.1766](https://doi.org/10.1002/SEC.1766).
- [54] Adam O’Neill. *Definitional Issues in Functional Encryption*. Cryptology ePrint Archive, Report 2010/556. 2010. URL: <https://eprint.iacr.org/2010/556>.
- [55] Rafael Pass. “On Deniability in the Common Reference String and Random Oracle Model”. In: *CRYPTO 2003*. Ed. by Dan Boneh. Vol. 2729. LNCS. Springer, Berlin, Heidelberg, Aug. 2003, pp. 316–337. DOI: [10.1007/978-3-540-45146-4\\_19](https://doi.org/10.1007/978-3-540-45146-4_19).
- [56] Seyoon Ragavan, Neekon Vafa, and Vinod Vaikuntanathan. “Indistinguishability Obfuscation from Bilinear Maps and LPN Variants”. In: *TCC 2024, Part IV*. Ed. by Elette Boyle and Mohammad Mahmoody. Vol. 15367. LNCS. Springer, Cham, Dec. 2024, pp. 3–36. DOI: [10.1007/978-3-031-78023-3\\_1](https://doi.org/10.1007/978-3-031-78023-3_1).
- [57] Ronald L. Rivest, Adi Shamir, and Yael Tauman. “How to Leak a Secret”. In: *ASIACRYPT 2001*. Ed. by Colin Boyd. Vol. 2248. LNCS. Springer, Berlin, Heidelberg, Dec. 2001, pp. 552–565. DOI: [10.1007/3-540-45682-1\\_32](https://doi.org/10.1007/3-540-45682-1_32).
- [58] Michal Rjaško and Martin Stanek. *On Designated Verifier Signature Schemes*. Cryptology ePrint Archive, Report 2010/191. 2010. URL: <https://eprint.iacr.org/2010/191>.
- [59] Shahrokh Saeednia, Steve Kremer, and Olivier Markowitch. “An Efficient Strong Designated Verifier Signature Scheme”. In: *ICISC 03*. Ed. by Jong In Lim and Dong Hoon Lee. Vol. 2971. LNCS. Springer, Berlin, Heidelberg, Nov. 2004, pp. 40–54. DOI: [10.1007/978-3-540-24691-6\\_4](https://doi.org/10.1007/978-3-540-24691-6_4).
- [60] Amit Sahai and Brent Waters. “How to use indistinguishability obfuscation: deniable encryption, and more”. In: *46th ACM STOC*. Ed. by David B. Shmoys. ACM Press, 2014, pp. 475–484. DOI: [10.1145/2591796.2591825](https://doi.org/10.1145/2591796.2591825).
- [61] Daniel Shanks. “Class number, a theory of factorization, and genera”. In: *1969 Number Theory Institute*. Ed. by Donald J. Lewis. Vol. 20. Proceedings of Symposia in Pure Mathematics. Providence, Rhode Island: American Mathematical Society, 1971, pp. 415–440. ISBN: 0-8218-1420-6. DOI: [10.1090/pspum/020/0316385](https://doi.org/10.1090/pspum/020/0316385).

- [62] Kyung-Ah Shim. “Rogue-key attacks on the multi-designated verifiers signature scheme”. In: *Inf. Process. Lett.* 107.2 (2008), pp. 83–86. DOI: [10.1016/J.IPL.2007.11.021](https://doi.org/10.1016/j.ipl.2007.11.021).
- [63] Victor Shoup. “Lower Bounds for Discrete Logarithms and Related Problems”. In: *EUROCRYPT’97*. Ed. by Walter Fumy. Vol. 1233. LNCS. Springer, Berlin, Heidelberg, May 1997, pp. 256–266. DOI: [10.1007/3-540-69053-0\\_18](https://doi.org/10.1007/3-540-69053-0_18).
- [64] Michael A. Specter, Sunoo Park, and Matthew Green. “KeyForge: Non-Attributable Email from Forward-Forgeable Signatures”. In: *USENIX Security 2021*. Ed. by Michael Bailey and Rachel Greenstadt. USENIX Association, Aug. 2021, pp. 1755–1773. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/specter-keyforge>.
- [65] Xi Sun, Haibo Tian, and Yumin Wang. “Toward Quantum-Resistant Strong Designated Verifier Signature from Isogenies”. In: *2012 Fourth International Conference on Intelligent Networking and Collaborative Systems, INCoS 2012, Bucharest, Romania, September 19-21, 2012*. Ed. by Fatos Xhafa, Leonard Barolli, Florin Pop, Xiaofeng Chen, and Valentin Cristea. IEEE, 2012, pp. 292–296. DOI: [10.1109/INCOS.2012.70](https://doi.org/10.1109/INCOS.2012.70).
- [66] P. Thanalakshmi, R. Anitha, N. Anbazhagan, Chulho Park, Gyanendra Prasad Joshi, and Changho Seo. “A Hash-Based Quantum-Resistant Designated Verifier Signature Scheme”. In: *Mathematics* 10.10 (2022). ISSN: 2227-7390. DOI: [10.3390/math10101642](https://doi.org/10.3390/math10101642).
- [67] Haibo Tian, Xiaofeng Chen, and Jin Li. “A Short Non-delegatable Strong Designated Verifier Signature”. In: *ACISP 12*. Ed. by Willy Susilo, Yi Mu, and Jennifer Seberry. Vol. 7372. LNCS. Springer, Berlin, Heidelberg, July 2012, pp. 261–279. DOI: [10.1007/978-3-642-31448-3\\_20](https://doi.org/10.1007/978-3-642-31448-3_20).
- [68] Raylin Tso, Takeshi Okamoto, and Eiji Okamoto. “Practical Strong Designated Verifier Signature Schemes Based on Double Discrete Logarithms”. In: *Information Security and Cryptology, First SKLOIS Conference, CISC 2005, Beijing, China, December 15-17, 2005, Proceedings*. Ed. by Dengguo Feng, Dongdai Lin, and Moti Yung. Vol. 3822. Lecture Notes in Computer Science. Springer, 2005, pp. 113–127. DOI: [10.1007/11599548\\_10](https://doi.org/10.1007/11599548_10).
- [69] Nik Unger and Ian Goldberg. “Deniable Key Exchanges for Secure Messaging”. In: *ACM CCS 2015*. Ed. by Indrajit Ray, Ninghui Li, and Christopher Kruegel. ACM Press, Oct. 2015, pp. 1211–1223. DOI: [10.1145/2810103.2813616](https://doi.org/10.1145/2810103.2813616).
- [70] Nik Unger and Ian Goldberg. “Improved Strongly Deniable Authenticated Key Exchanges for Secure Messaging”. In: *PoPETs 2018.1* (Jan. 2018), pp. 21–66. DOI: [10.1515/popets-2018-0003](https://doi.org/10.1515/popets-2018-0003).
- [71] Nihal Vatandas, Rosario Gennaro, Bertrand Ithurburn, and Hugo Krawczyk. “On the Cryptographic Deniability of the Signal Protocol”. In: *ACNS 2020, Part II*. Ed. by Mauro Conti, Jianying Zhou, Emiliano Casalichio, and Angelo Spognardi. Vol. 12147. LNCS. Springer, Cham, Oct. 2020, pp. 188–209. DOI: [10.1007/978-3-030-57878-7\\_10](https://doi.org/10.1007/978-3-030-57878-7_10).
- [72] Fenghe Wang, Yupu Hu, and Baocang Wang. “Lattice-based strong designate verifier signature and its applications”. In: *Malaysian Journal of Computer Science* 25 (1 2012), pp. 11–22.
- [73] Yunmei Zhang, Man Ho Au, Guomin Yang, and Willy Susilo. “(Strong) Multi-Designated Verifiers Signatures Secure against Rogue Key Attack”. In: *Network and System Security - 6th International Conference, NSS 2012, Wuyishan, Fujian, China, November 21-23, 2012. Proceedings*. Ed. by Li Xu, Elisa Bertino, and Yi

Mu. Vol. 7645. Lecture Notes in Computer Science. Springer, 2012, pp. 334–347.  
DOI: [10.1007/978-3-642-34601-9\\_25](https://doi.org/10.1007/978-3-642-34601-9_25).

|                                                                                     |                                                                                                |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <u>Setup'():</u>                                                                    | <u>Sign'(pp, sk<sub>S</sub>, pk<sub>D</sub>  a, id<sub>D</sub>, m):</u>                        |
| 1 <b>return</b> Setup()                                                             | 5 $\sigma \xleftarrow{\$} \text{Sign}(\text{pp}, \text{sk}_S, \text{pk}_D, \text{id}_D, m)$    |
| <u>SKGen'(pp):</u>                                                                  | 6 <b>if</b> $a = 1$ <b>then return</b> $\sigma    \text{sk}_S$                                 |
| 2 <b>return</b> SKGen(pp)                                                           | 7 <b>return</b> $\sigma$                                                                       |
| <u>VKGen'(pp, id<sub>D</sub>):</u>                                                  | <u>Sim'(pp, pk<sub>S</sub>, sk<sub>D</sub>, id<sub>D</sub>, m):</u>                            |
| 3 $(\text{pk}_D, \text{sk}_D) \xleftarrow{\$} \text{VKGen}(\text{pp}, \text{id}_D)$ | 8 <b>return</b> Sim(pp, pk <sub>S</sub> , sk <sub>D</sub> , id <sub>D</sub> , m)               |
| 4 <b>return</b> $(\text{pk}_D    0, \text{sk}_D)$                                   | <u>Vrfy'(pp, pk<sub>S</sub>, pk<sub>D</sub>, id<sub>D</sub>, m, <math>\sigma    b</math>):</u> |
|                                                                                     | 9 <b>return</b> Vrfy(pp, pk <sub>S</sub> , pk <sub>D</sub> , id <sub>D</sub> , m, $\sigma$ )   |

**Fig. 11.** A DVS scheme  $\text{DVS}' = (\text{Setup}', \text{SKGen}', \text{VKGen}', \text{Sign}', \text{Sim}', \text{Vrfy}')$  based on a DVS scheme  $\text{DVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{Sim}, \text{Vrfy})$  that is unforgeable and source hiding. Separates uf and mal-uf (see [Theorem 13](#)), as well as sh and s-mal-sh (see [Theorem 14](#)).

## A Separations of security notions for DVS

Here, we separate our new security notions for DVS against malicious verifiers from the existing ones against honest verifiers. While malicious unforgeability is strictly stronger than (honest) unforgeability, source hiding and simplified malicious source hiding are actually incomparable.

**DVS unforgeability.** We begin with separating unforgeability from malicious unforgeability.

**Theorem 13.** *Assume an  $(t, \varepsilon, q_S, q_V, q_{VK})$ -unforgeable DVS scheme exists, then there exists a DVS scheme that is  $(t, \varepsilon, q_S, q_V, q_{VK})$ -unforgeable and not  $(t', \varepsilon', q'_S, q'_V, q'_{VK})$ -malicious unforgeable for arbitrary  $t', q'_V, q'_{VK}$ , and  $q'_S \geq 1$  and  $\varepsilon' < 1$ .*

*Proof.* Given an unforgeable DVS scheme  $\text{DVS}$ , we build  $\text{DVS}'$  as shown in [Figure 11](#):  $\text{VKGen}'$  appends a zero bit to the public key.  $\text{Sign}'$  checks the extra bit of the verifier's public key: If it is 1, it appends  $\text{sk}_S$  to the signature. All other algorithms ignore the extra bit and work the same way as  $\text{DVS}$ .

This scheme retains unforgeability of  $\text{DVS}$  since the adversary cannot trigger the special condition in  $\text{Sign}'$  within the  $\text{SIGN}$  oracle with honestly generated verifier keys.

However, this scheme is not malicious unforgeable: The adversary can run  $\text{VKGen}'(\text{pp}, \text{id}_D)$  (for some  $\text{id}_D \in \mathcal{IDS}$ ) to obtain  $(\text{pk}_D || 0, \text{sk}_D)$  and query the signing oracle on the public key and identity  $\text{pk}_D || 1, \text{id}_D$  for any message. Thereby it learns the signer's secret key and can sign any message from the challenge signer designated for the challenge verifier.  $\square$

On the other hand, malicious unforgeability implies (honest) unforgeability since any adversary that breaks (honest) unforgeability also breaks malicious unforgeability.

**DVS source hiding.** Next, we show that source hiding and simplified malicious source hiding are incomparable. On a technical note, we additionally need to assume that it is not feasible to compute secret keys from public keys (and other public information) in the first separation. Observe that this is a reasonable assumption since otherwise anybody can forge signatures.

**Theorem 14.** *Assume a  $(t, \varepsilon, q_C)$ -source hiding DVS scheme exists for which it is not feasible to compute the secret key from public information (including the public key), then there exists a DVS scheme that is  $(t, \varepsilon, q_C)$ -source hiding and not  $(t_A, t_E, t_D, \varepsilon', q_{VK})$ -simplified malicious source hiding for arbitrary  $t_A, t_E, t_D, q_R$ , and  $\varepsilon' < 1$ .*

*Proof.* Once more, we use the scheme shown in Figure 11, which appends a 0 to honestly generated public keys and where signing designated for verifier public keys with a trailing 1 leaks the signer secret key.

The scheme retains source hiding of DVS since the adversary cannot trigger the special condition in  $\text{Sign}'$  within the CHALL oracle with honestly generated verifier keys.

However, the scheme is not simplified malicious source hiding: The adversary can run  $\text{VKGen}'(\text{pp}, \text{id}_D)$  (for some  $\text{id}_D \in \mathcal{IDS}$ ) to obtain  $(\text{pk}_D || 0, \text{sk}_D)$  and query the challenge oracle on the public key and identity  $\text{pk}_D || 1, \text{id}_D$  for any message. Due to the trailing 1 bit of the public key, the signature now has the signer secret key appended. For  $b = 1$ , the extractor needs to produce a signature with  $\text{sk}_S$  appended, even though the adversary does not know  $\text{sk}_S$ . Hence, if the extractor succeeds, it must be able to compute the secret key  $\text{sk}_S$  from public information, contradicting our assumption.  $\square$

The perhaps surprising separation from simplified malicious source hiding to source hiding can intuitively be explained with the source hiding adversary having access to the challenge signer secret key  $\text{sk}_S$  and the challenge oracle CHALL at the same time, which neither the adversary nor the distinguisher in the s-mal-sh game have.

**Theorem 15.** *Assume a  $(t_A, t_E, t_D, \varepsilon, q_{VK})$ -simplified malicious source hiding DVS scheme exists, then there exists a DVS scheme that is  $(t_A, t_E, t_D, \varepsilon - 2^{-\lambda}, q_{VK})$ -simplified malicious source hiding and for all possible choices of  $\text{Sim}'$  not  $(t, 3/4, 1)$ -source hiding for arbitrary  $t$  and for any  $\lambda \in \mathbb{N}$ .*

*Proof.* Given a simplified malicious source hiding DVS scheme DVS we construct a new DVS scheme  $\text{DVS}'$  as described in Figure 12: Key generation appends a randomly sampled bit string  $r$  of length  $\lambda$  to the secret key and when calling  $\text{Sign}'$  with the signer secret key as message the algorithm appends a 1 bit to the signature and otherwise a 0 bit and  $\text{Vrfy}'$  ignores the extra bit.

The scheme  $\text{DVS}'$  retains simplified malicious source hiding of DVS. The extractor for  $\text{DVS}'$  runs the extractor for DVS on its own input and returns the resulting signature with an appended 0 bit. Since the view of the adversary is independent of  $r$ , the probability that it made a query where  $\text{Sign}'$  would append a 1 bit is at most  $2^{-\lambda}$ .

|                                                                                                                                                                                             |                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <u>Setup'():</u><br>1 <b>return</b> Setup()                                                                                                                                                 | <u>Sign'(pp, sk'_S = (sk_S, r), pk_D, id_D, m):</u><br>6 $\sigma \xleftarrow{\$} \text{Sign}(\text{pp}, \text{sk}_S, \text{pk}_D, \text{id}_D, m)$<br>7 <b>if</b> $m = \text{sk}'_S$ <b>then return</b> $\sigma    1$<br>8 <b>return</b> $\sigma    0$ |
| <u>SKGen'(pp):</u><br>2 $(\text{pk}_S, \text{sk}_S) \xleftarrow{\$} \text{SKGen}(\text{pp})$<br>3 $r \xleftarrow{\$} \{0, 1\}^\lambda$<br>4 <b>return</b> $(\text{pk}_S, (\text{sk}_S, r))$ | <u>Vrfy'(pp, pk_S, sk_D, id_D, m, <math>\sigma    b</math>):</u><br>9 <b>return</b> Vrfy(pp, pk_S, sk_D, id_D, m, $\sigma$ )                                                                                                                           |
| <u>VKGen'(pp, id_D):</u><br>5 <b>return</b> VKGen(pp, id_D)                                                                                                                                 |                                                                                                                                                                                                                                                        |

**Fig. 12.** A DVS scheme  $\text{DVS}' = (\text{Setup}', \text{SKGen}', \text{VKGen}', \text{Sign}', \text{Vrfy}')$  that is simplified malicious source hiding but not source hiding, based on a DVS scheme  $\text{DVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{Sim}, \text{Vrfy})$  that is simplified malicious source hiding (see [Theorem 15](#)). In [Theorem 15](#) we show that no algorithm  $\text{Sim}'$  can exist.

However, the scheme  $\text{DVS}'$  is not source hiding: The adversary first obtains a verifier key with the VKEY oracle<sup>11</sup>. Next, it parses the signer secret key as  $(\text{sk}_S, r) \leftarrow \text{sk}'_S$ , samples  $r^* \xleftarrow{\$} \{0, 1\}^\lambda$ , and sets  $\text{sk}_S^* \leftarrow (\text{sk}_S, r^*)$ . Now, the adversary flips a bit  $b'' \xleftarrow{\$} \{0, 1\}$  and queries the CHALL oracle on the messages  $\text{sk}'_S$  if  $b'' = 1$  or  $\text{sk}_S^*$  if  $b'' = 0$ . The adversary outputs the guess 0 if the last bit of the signature received from CHALL matches  $b''$  and 1 otherwise.

If the game is played with  $b = 0$  (where CHALL uses  $\text{DVS}'.\text{Sign}'$ ), the adversary always outputs the correct bit. If the game is played with  $b = 1$ , the inputs to the  $\text{Sim}'$  algorithm are statistically independent of  $r$  and thus  $\text{sk}'_S$  and  $\text{sk}_S^*$  are identically distributed given only the inputs of  $\text{Sim}'$ . Thus, the last bit of the signature it outputs is statistically independent of  $b''$  and the adversary guesses  $b''$  correctly with probability  $3/4$ .  $\square$

## B On the deniability of Signal's initial handshake X3DH

In [Section 5.2](#) we show that the EKD assumption is broken. Here, we use the same strategy to show that Signal's (former) initial handshake protocol X3DH is not deniable if the auxiliary input contains a hint on a public key, and the hinted CDH problem is hard for an algorithm bound to the runtime of the simulator (roughly corresponding to the extractor in our simplified malicious source hiding definition), but not for an algorithm bound to the runtime of the distinguisher.

Let us briefly review the X3DH handshake with the help of [Figure 13](#). All parties have a long-term DH key pair, several semi-static DH key pairs (which are used for several sessions each), and ephemeral DH key pairs (each used only once). Furthermore, parties have a long-term signing key, which they use to authenticate their semi-static public keys. First, Bob builds a so-called pre-key bundle, which is peer independent, and consists of his long-term public keys, his

<sup>11</sup> Keep in mind that this oracle is provided by the sh game, whereas the s-mal-sh game provides a VKEYS oracle.

**KGenLT:**

```

1 (ltpkUDH, ltskUDH) ←$ DH.KGen()
2 (ltpkUΣ, ltskUΣ) ←$ Σ.KeyGen()
3 return ((ltpkUDH, ltpkUΣ), (ltskUDH, ltskUΣ))

```

**Alice****KGenSS:**

```

4 (sspkUDH, ssskUDH) ←$ DH.KGen()
5 (sspkUKEM, ssskUKEM) ←$ KEM.KGen()
6 return ((sspkUDH, sspkUKEM), (ssskUDH, ssskUKEM))

```

**Bob**

$m_1$

←-----

```

Run(ltskA, ssskA, ltpk, sspk, πA, m1)
(epkADH, eskADH) ←$ DH.KGen()
(B, ssid, σDHssid, epkB, σKEM) ← m1
(sspkBDH, sspkBKEM) ← sspkBssid
(epkBDH, epkBKEM) ← epkB
if Σ.Ver(ltpkB, sspkBDH, σDHssid) = false
  return (πA, ε, ε)
DH1 ← DH(ltskA, sspkBDH)
DH2 ← DH(eskADH, ltpkB)
DH3 ← DH(eskADH, sspkBDH)
if epkBDH ≠ ⊥ //ephemeral DH key present
  DH4 ← DH(eskADH, epkBDH)
else
  DH4 ← ε
if epkBKEM ≠ ⊥ //ephemeral KEM key present
  if Σ.Ver(ltpkB, epkBKEM, σKEM) = false
    return (πA, ε, ε)
  (ct, ss) ← KEM.Encaps(epkBKEM)
else //no ephemeral KEM key present
  if Σ.Ver(ltpkB, sspkBKEM, σKEM) = false
    return (πA, ε, ε)
  (ct, ss) ← KEM.Encaps(sspkBKEM)
ms ← DH1||DH2||DH3||DH4||ss
πA.K ← KDF(ms)
πA.pid ← B
m2 ← (A, epkADH, ct)
return (πA, m2)

```

Run(ltsk<sub>B</sub>, ltpk, sssk<sub>B</sub>, sspk, π<sub>B</sub>, m<sub>0</sub>)

```

(create, (ssid, eDH, eKEM)) ← m0
πB.pid ← *
(sspkBDH, sspkBKEM) ← sspkBssid
if σDHssid = ⊥ //saved from a previous run?
  σDHssid ←$ Sign(ltskB, sspkBDH)
if eDH = true
  (epkBDH, eskBDH) ←$ DH.KeyGen()
else
  epkBDH ← ⊥
  if eKEM = true
    (epkBKEM, eskBKEM) ←$ KEM.KeyGen()
    σKEM ←$ Sign(ltskB, epkBKEM)
  else
    if σKEMssid = ⊥ //saved from a previous run?
      σKEMssid ←$ Sign(ltskB, sspkBKEM)
      σKEM ← σKEMssid
      epkBKEM ← ⊥
    epkB ← (epkBDH, epkBKEM)
  m1 ← (B, ssid, σDHssid, epkB, σKEM)
  return (πB, m1)

```

$m_2$

-----→

```

Run(ltskB, ssskB, ltpk, sspk, πB, m2)
(ssskBDH, ssskBKEM) ← ssskBssid
(A, epkADH, ct) ← m2
DH1 ← DH(ltpkA, ssskBDH)
DH2 ← DH(epkADH, ltskB)
DH3 ← DH(epkADH, ssskBDH)
if epkBDH ≠ ⊥ //ephemeral DH key present
  DH4 ← DH(epkADH, eskBDH)
else
  DH4 ← ε
if epkBKEM ≠ ⊥ //ephemeral KEM key
  ss ← KEM.Decaps(eskBKEM, ct)
else //no ephemeral KEM key present
  ss ← KEM.Decaps(ssskBKEM, ct)
ms ← KDF(DH1||DH2||DH3||DH4||ss)
πB.K ← KDF(ms)
πB.pid ← A
return (πB, ε)

```

**Fig. 13.** Signal's initial handshake protocols X3DH and PQXDH, following the presentation of [32, Figure 3] (but without the AEAD and user messages). The KEM with gray background is exclusive to PQXDH.

semi-static public key including the signature, and, optionally, his ephemeral public key. Upon receiving Bob’s pre-key bundle, Alice verifies the signature and computes several DH combinations: long-term–semi-static, ephemeral–long-term, ephemeral–semi-static, and, if Bob’s pre-key bundle contains an ephemeral key, ephemeral–ephemeral. She uses KDF to derive the session key from these four DH shared secrets and sends her ephemeral public key to Bob, who can compute the session key in the same way. Alice’s message can be combined with an AEAD ciphertext of her first user message (such as “Hi Bob, how’re you doing?”) encrypted under the session key<sup>12</sup>. In this case, Bob decrypts the AEAD ciphertext and aborts if decryption fails. Whether this AEAD ciphertext is part of the initial handshake is ambiguous [51, 45]. Vatandas et al. [71] did not model the AEAD ciphertext as part of the initial handshake<sup>13</sup>, while [32] did. We do not consider the AEAD as part of the initial handshake.

Our result contrasts the result of [71], which shows deniability assuming the EKDH assumption without restricting the admissible auxiliary inputs. We embed a hinted CDH challenge as auxiliary input and leverage the difference in runtime between simulator and distinguisher to exclude and ensure breaking the hinted CDH assumption with different parameters. Though, this does not invalidate the results of [32], since they use a specific class of auxiliary inputs, i.e., one pre-key bundle per user and semi-static key<sup>14</sup>. This does not include a hint on a public key and therefore we cannot reduce deniability of X3DH to breaking the hinted CDH assumption for this class of auxiliary inputs.

Our attack relies on embedding a hinted CDH challenge and not on the presence or absence of the AEAD ciphertext in Alice’s message. Hence, we expect our attack to transfer even if one considers the AEAD ciphertext as part of Alice’s message, assuming the AEAD scheme achieves INT-CTXT.

Our attack also applies to the deniability of PQXDH if the auxiliary inputs lend themselves to building a hinted CDH challenge. For auxiliary inputs that do not facilitate forming a hinted CDH challenge our results do not rule out deniability, as discussed above.

We borrow the deniability definition of Vatandas et al. [71, Definition 5], which adapts the definition of Di Raimondo et al. [23, Definition 2] to concrete security, and we adapt it further for syntactical differences and more detailed treatment of auxiliary inputs. In particular, both prior definitions have quantified the auxiliary input before the simulator. This allows the simulator to arbitrarily depend on the auxiliary input, including on all secret keys corresponding to public keys used in the auxiliary input. Furthermore, both definitions first quantify auxiliary inputs and then sample keys. However, inspection of their proofs suggests that the

<sup>12</sup> More specifically, encrypted under a key derived from the session key. But the details of Signal’s key scheduling do not affect our results.

<sup>13</sup> They consider only two different lifetimes for Bob as well; it is not immediately obvious if they dropped semi-static or ephemeral keys, as discussed in [32].

<sup>14</sup> They use the auxiliary input to ensure that a signature is available when faking a transcript. They also have a second setting in which the Fake algorithm learns the signature from an oracle. In this second setting they do not rely on auxiliary inputs.



auxiliary input depends on the sampled keys. Hence, we use a sampling algorithm for auxiliary inputs  $\text{AuxPrep}$  that depends on the sampled keys. This auxiliary input sampled with  $\text{AuxPrep}$ , called  $\text{aux}_1$  below, can be used to help the simulator. In contrast, we add a second auxiliary input that can help the adversary to the disadvantage of the simulator. This second input, called  $\text{aux}_2$ , models arbitrary data that the adversary may have access to, following our approach for simplified malicious source hiding of DVS (cp. [Definition 19](#)). We will use this  $\text{aux}_2$  to embed a public key for the hinted CDH challenge that, allowing us to disprove deniability of X3DH. Though, the basic idea, including that the adversary's view from interacting with an oracle is compared to a simulated view, remains the same.

**Definition 22 (Deniable Key Exchange (adapted from [23, 71])).** *An AKE protocol  $\Pi = (\text{KGenLT}, \text{KGenSS}, \text{Run})$  is a  $(t_A, t_S, t_D, \varepsilon_D)$ -concurrently deniable key exchange protocol wrt. auxiliary input sampler  $\text{AuxPrep}$  if for any adversary  $\mathcal{A}$  running in time  $t_A$ , there exists a simulator  $\text{SIM}$  running in time  $t_S$ , such that for the two distributions*

| <i>Real():</i>                                                                                                                                    | <i>Sim():</i>                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 $\mathcal{L}_{\text{pk}} \leftarrow \emptyset; \mathcal{L}_{\text{sk}} \leftarrow \emptyset$                                                    | 11 $\mathcal{L}_{\text{pk}} \leftarrow \emptyset; \mathcal{L}_{\text{sk}} \leftarrow \emptyset$                                                    |
| 2 <b>for</b> $U \in [n_p]$                                                                                                                        | 12 <b>for</b> $U \in [n_p]$                                                                                                                        |
| 3 $(\text{ltpk}_U, \text{ltsk}_U) \xleftarrow{\$} \text{KGenLT}()$                                                                                | 13 $(\text{ltpk}_U, \text{ltsk}_U) \xleftarrow{\$} \text{KGenLT}()$                                                                                |
| 4 <b>for</b> $\text{ssid} \in [n_{ss}]$                                                                                                           | 14 <b>for</b> $\text{ssid} \in [n_{ss}]$                                                                                                           |
| 5 $(\text{sspk}_U^{\text{ssid}}, \text{sssk}_U^{\text{ssid}}) \xleftarrow{\$} \text{KGenSS}()$                                                    | 15 $(\text{sspk}_U^{\text{ssid}}, \text{sssk}_U^{\text{ssid}}) \xleftarrow{\$} \text{KGenSS}()$                                                    |
| 6 $\mathcal{L}_{\text{pk}} \leftarrow \mathcal{L}_{\text{pk}} \cup \{\text{ltpk}_U, \{\text{sspk}_U^{\text{ssid}}\}_{\text{ssid} \in [n_{ss}]}\}$ | 16 $\mathcal{L}_{\text{pk}} \leftarrow \mathcal{L}_{\text{pk}} \cup \{\text{ltpk}_U, \{\text{sspk}_U^{\text{ssid}}\}_{\text{ssid} \in [n_{ss}]}\}$ |
| 7 $\mathcal{L}_{\text{sk}} \leftarrow \mathcal{L}_{\text{sk}} \cup \{\text{ltsk}_U, \{\text{sssk}_U^{\text{ssid}}\}_{\text{ssid} \in [n_{ss}]}\}$ | 17 $\mathcal{L}_{\text{sk}} \leftarrow \mathcal{L}_{\text{sk}} \cup \{\text{ltsk}_U, \{\text{sssk}_U^{\text{ssid}}\}_{\text{ssid} \in [n_{ss}]}\}$ |
| 8 $\text{aux}_1 \xleftarrow{\$} \text{AuxPrep}(\mathcal{L}_{\text{pk}}, \mathcal{L}_{\text{sk}})$                                                 | 18 $\text{aux}_1 \xleftarrow{\$} \text{AuxPrep}(\mathcal{L}_{\text{pk}}, \mathcal{L}_{\text{sk}})$                                                 |
| 9 $\text{aux} \leftarrow (\text{aux}_1, \text{aux}_2)$                                                                                            | 19 $\text{aux} \leftarrow (\text{aux}_1, \text{aux}_2)$                                                                                            |
| 10 <b>return</b> $(\text{aux}, \mathcal{L}_{\text{pk}}, \text{View}(\mathcal{A}(\mathcal{L}_{\text{pk}}, \text{aux})))$                           | 20 <b>return</b> $(\text{aux}, \mathcal{L}_{\text{pk}}, \text{SIM}(\mathcal{L}_{\text{pk}}, \text{aux}))$                                          |
| <i>we have for all probabilistic polynomial time <math>\mathcal{D}</math> and all <math>\text{aux}_2 \in \{0, 1\}^*</math></i>                    |                                                                                                                                                    |

$$\left| \Pr_{x \in \text{Real}()} [\mathcal{D}(x) = 1] - \Pr_{x \in \text{Sim}()} [\mathcal{D}(x) = 1] \right| \leq \varepsilon_D.$$

Equipped with this definition we proceed to show that deniability of X3DH does not hold under our [Definition 22](#) if we leverage the auxiliary input to pass a hinted CDH challenge to the adversary and appropriate parameters.

**Theorem 16.** *If  $\mathcal{G} = (\mathbb{G}, g, q)$  is a group where the  $(t, \varepsilon, \eta)$ -hinted CDH assumption holds for  $\varepsilon < 1$ , but the  $(t', \varepsilon', \eta)$ -hinted CDH assumption (for some  $t < t'$ ) does not hold, and we model KDF as random oracle, then the X3DH protocol over  $\mathcal{G}$  as described in [Figure 13](#) is not  $(t_A, t_S, t_D, \varepsilon_D)$ -deniable wrt. auxiliary inputs sampled with any  $\text{AuxPrep}$  for  $t_A$  small,  $t_S \approx t$ ,  $t_D \approx t'$  and  $\varepsilon_D \leq \varepsilon' - \varepsilon/2$ .*

*Proof.* For the real distribution, the adversary  $\mathcal{A}(\mathcal{L}_{\text{pk}}, \text{aux} = (\text{aux}_1, (g^z, \lfloor z/\eta \rfloor)))$  can set  $g^z$  as long-term public key for some party Alice, sample  $\text{esk}_A = x \xleftarrow{\$} \mathcal{G}$ , and compute  $\text{epk}_A \leftarrow g^x$  as an ephemeral public key. Next, it runs a session between Alice with ephemeral public key  $\text{epk}_A$  and some party Bob, where the

adversary controls Alice and accesses its oracle for Bob. After this one session, the adversary terminates. In the simulated distribution, the simulator has to produce indistinguishable output upon receiving the same input and without oracle access.

The distinguisher now gets the auxiliary input  $\mathbf{aux}$ , the list of public keys  $\mathcal{L}_{\text{pk}}$ , the transcript of messages, the session key(s) computed by the oracle, and the randomness of the adversary. The distinguisher can first run the adversary  $\mathcal{A}_{\text{HCDH}'}$  against the  $(t', \varepsilon', \eta)$ -hinted CDH assumption on input  $\mathbf{aux}_{\text{HCDH}} = (\mathbf{sspk}_B, g^z, \lfloor z/\eta \rfloor)$ , where  $\mathbf{sspk}_B$  is Bob's semi-static public key in the session above. The hinted CDH adversary  $\mathcal{A}_{\text{HCDH}'}$  outputs  $\widetilde{\text{DH}}_1$ , and the distinguisher can compute  $\text{DH}_2, \text{DH}_3, \text{DH}_4$  with the knowledge of Alice's ephemeral secret key  $\mathbf{esk}_A$ , which it obtains from the adversary's randomness. Finally, the distinguisher recomputes the session key as  $\tilde{K} \leftarrow \text{KDF}(\widetilde{\text{DH}}_1, \text{DH}_2, \text{DH}_3, \text{DH}_4)$ . If  $\tilde{K}$  is identical to the session key provided to the distinguisher as input, it outputs 0, otherwise 1.

The runtime of the distinguisher is dominated by the runtime of the hinted CDH adversary  $t'$ , since all it otherwise does is getting  $\mathbf{esk}_A$  from the randomness of  $\mathcal{A}$  as well as computing DH and KDF.

If the distinguisher got a sample from the real distribution, it answers correctly iff the hinted CDH adversary  $\mathcal{A}_{\text{HCDH}'}$  succeeds, i.e., with probability  $> \varepsilon'$ . Next, observe that the simulator runs in time  $t_S \approx t$  and, by the  $(t, \varepsilon, \eta)$ -hinted CDH assumption, can compute  $\widetilde{\text{DH}}_1$  with probability  $\leq \varepsilon$ . If the distinguisher got a sample from the simulated distribution, it answers correctly iff  $\mathcal{A}_{\text{HCDH}'}$  succeeds and the simulator fails to compute  $\widetilde{\text{DH}}_1$ : If both the simulator and  $\mathcal{A}_{\text{HCDH}'}$  succeed, then the distinguisher falsely classifies the sample as real, which happens with a probability  $\leq \varepsilon$ . If only  $\mathcal{A}_{\text{HCDH}'}$  succeeds, the distinguisher correctly classifies the sample as simulated: Since the simulator does not know  $\widetilde{\text{DH}}_1$  it cannot learn the session key, which is a random oracle output on values including  $\text{DH}_1$ . This happens with probability  $> \varepsilon' - \varepsilon$ . All in all, the distinguisher succeeds with probability  $> \varepsilon' - \varepsilon/2$ .

The same proof strategy works for a malicious Bob: Embed the hinted CDH challenge as ephemeral, semi-static, or long-term key and sample the other two key pairs honestly (so that the distinguisher can learn those two secrets). If the challenge is embedded as Alice's ephemeral key (or as Bob's semi-static key, respectively), the distinguisher needs to solve three (or two, respectively) hinted CDH challenges, since these keys are used for more than one DH operation. In that case the distinguisher would need to run the hinted CDH adversary  $\mathcal{A}_{\text{HCDH}'}$  two or three times. We can account for that with  $t_D \approx 3t'$ . Similarly, the winning probability now needs to take into account that the hinted CDH adversary is run more than once. To put it in a nutshell, we can embed the hinted public key as any key of Alice or Bob. Embedding the key in Alice's long-term, Bob's long-term, or Bob's ephemeral key results in an attack with the given parameters. When embedding the key in Alice's ephemeral or Bob's semi-static key, we may need to allow for a longer runtime of the distinguisher or worse advantage.  $\square$

## C Omitted lemma

**Lemma 11.** *For  $\varepsilon \in [0, 1)$ ,  $\delta \in [0, 1 - \varepsilon]$  we have  $\varepsilon/(1 - \delta) \leq \varepsilon + \delta$ .*

*Proof.* The inequality  $\varepsilon/(1 - \delta) \leq \varepsilon + \delta$  can be rearranged to  $0 \leq \delta - \varepsilon\delta - \delta^2 = \delta(1 - \varepsilon - \delta)$ . Clearly the inequality holds in the edge cases  $\delta = 0$  and  $\delta = 1 - \varepsilon$ . It also holds for all  $\delta \in (0, 1 - \varepsilon)$ : because the function

$$\begin{aligned} f : \mathbb{R} &\rightarrow \mathbb{R} \\ \delta &\mapsto \delta - \varepsilon\delta - \delta^2 \end{aligned}$$

is continuous, there exists a  $\delta \in (0, 1 - \varepsilon)$  such that  $f(\delta) > 0$  (this can be seen for example by observing that the derivative  $f'(\delta) = 1 - \varepsilon - 2\delta$  is positive for  $\delta = 0$ ), and  $\delta \in \{0, 1 - \varepsilon\}$  are the only roots of  $f$ . Under these conditions the intermediate value theorem guarantees  $f(\delta) > 0$  for  $\delta \in (0, 1 - \varepsilon)$ .  $\square$